Contents

Human in the Loop Machine Learning **1**
Misunderstood . . . 1
What This Handbook Covers . . . 1
A Note on Code . . . 2
Notation . . . 2

Declaration of Originality **3**

Abstract **5**

Acknowledgments **7**

What Is Human-in-the-Loop Machine Learning? **9**
The Automation Paradox . . . 9
Defining Human-in-the-Loop ML . . . 9
A Brief History . . . 10
Why HITL? The Case for Human Judgment . . . 11
The Feedback Loop . . . 11
What HITL Is Not . . . 12
The Economics of Human Feedback . . . 12
Overview of the Handbook . . . 13

A Taxonomy of Human–Machine Interaction **15**
Levels of Automation . . . 15
Active vs. Passive Human Involvement . . . 16
Single vs. Multiple Annotators . . . 16
A Unified Framework . . . 17

Data Annotation and Labeling **19**
Types of Annotation . . . 19
Annotation Guidelines . . . 20
Measuring Annotation Quality: Inter-Annotator Agreement . . . 20
Handling Disagreements . . . 22
Label Noise and Its Effects . . . 22
Annotation Cost and Throughput . . . 23

Active Learning **25**
The Active Learning Setup . . . 25
Theoretical Foundations . . . 25
Query Strategies . . . 26
Uncertainty Estimation for Deep Models . . . 27

A Full Active Learning Loop . . . 29
The Cold Start Problem . . . 30
Batch-Mode Active Learning . . . 31
Stopping Criteria . . . 31
When Active Learning Works (and When It Doesn't) . . . 31

Interactive Machine Learning **33**
Principles of Interactive Machine Learning . . . 33
Comparison with Active Learning . . . 33
Mixed-Initiative Interaction . . . 34
Human Factors in IML . . . 34
The IML Feedback Types in Practice . . . 35
Case Study: Google Teachable Machine . . . 35
Implementing a Simple IML Loop . . . 35
The Grandmother Test . . . 37
IML and Foundation Models . . . 38

Reinforcement Learning from Human Feedback **39**
The Alignment Problem . . . 39
The RLHF Pipeline . . . 39
A Simplified RLHF Demonstration . . . 40
Challenges in RLHF . . . 42
Constitutional AI (RLAIF) . . . 43

Learning from Demonstrations **45**
Behavioral Cloning . . . 45
Inverse Reinforcement Learning . . . 46
GAIL: Generative Adversarial Imitation Learning . . . 46
Behavioral Cloning in NLP: A Practical Example . . . 47
Comparison of Imitation Learning Methods . . . 48
Applications . . . 49

Learning from Comparisons and Rankings **51**
Why Comparisons Are Better Than Ratings . . . 51
The Bradley-Terry Model . . . 51
Thurstone's Model . . . 53
Rank Aggregation . . . 54
Dueling Bandits . . . 54
Preference Learning for Language Models . . . 54
Collecting High-Quality Preference Data . . . 54

HITL in Natural Language Processing **57**
The Constitutive Problem in NLP Annotation . . . 57
Text Classification Annotation . . . 58
Named Entity Recognition . . . 58
Relation Extraction and Semantic Annotation . . . 59
Machine Translation Post-Editing (MTPE) . . . 59
Conversational AI and Dialogue Annotation . . . 59
Programmatic Labeling and Weak Supervision . . . 60
RLHF for Language Models: The Annotation Reality . . . 61
The Annotation–Model Feedback Loop . . . 62
Evaluation of Generative NLP Models . . . 62

HITL in Computer Vision **63**
How Annotation Choices Become Model Biases . . . 63
Image Classification Annotation . . . 64
Object Detection: Bounding Box Annotation . . . 64
Semantic and Instance Segmentation . . . 65
Active Learning for Computer Vision . . . 66
Semi-Supervised Learning with Human Guidance . . . 67
Video Annotation . . . 68

HITL in Healthcare and Science **69**
Medical Image Analysis . . . 69
Clinical NLP Annotation . . . 70
Regulatory Considerations . . . 71
Scientific Data Annotation . . . 71
Managing Specialist Annotators . . . 72
A HITL Active Learning Pipeline for Medical Imaging . . . 73

Annotation Platforms and Tooling **77**
The Annotation Platform Landscape . . . 77
Annotation Platform Feature Comparison . . . 78
Annotation as Code . . . 78

Crowdsourcing and Quality Control **81**
Crowdsourcing Platforms . . . 81
Task Design for Crowdsourcing . . . 81
Statistical Models for Label Aggregation . . . 83
Redundancy and Aggregation Strategy . . . 85

Evaluation and Metrics **87**
Model-Centric Metrics . . . 87
Annotation Efficiency Metrics . . . 88
Human Evaluation . . . 89
Measuring Alignment with Human Intent . . . 90
A/B Testing in Deployed Systems . . . 91

Fairness, Bias, and Ethics **93**
Sources of Bias in HITL Systems . . . 93
Fairness in HITL Systems . . . 94
Annotator Welfare . . . 94
Privacy in Annotation . . . 95
Adversarial Annotation and Data Poisoning . . . 96
Institutional Ethics . . . 96

Future Directions **97**
Foundation Models and the Changing Role of Annotation . . . 97
LLMs as Annotators . . . 97
Weak Supervision at Scale . . . 98
Continual Learning with Human Oversight . . . 98
Multi-Modal HITL . . . 99
Scalable Oversight . . . 99
The Evolving Division of Labor . . . 99
Open Research Problems . . . 100

Bibliography **101**

Case Study: Limen — A Human in the Loop of Everything **103**
The Problem with Conventional OS Design 103
Architecture at the Threshold 104
The Interaction Design 105
Limen as a HITL System 106
The Deeper Point 106

On Leaving **109**

Human in the Loop Machine Learning

Misunderstood

> *There is no need for a PhD titled "Human in the Loop Machine Learning." Or rather — there is every need, and it has never been written. This is not that PhD. It is what comes instead.*

This handbook is a comprehensive, executable, and deliberately un-academic guide to **Human-in-the-Loop (HITL) Machine Learning** — the discipline of designing systems where human judgment and machine intelligence do not merely coexist but actively reshape each other.

It is *misunderstood* in at least three ways:

The field is misunderstood. Ask most people how machine learning works and they will describe a process that ends when the model is deployed. In reality, the human is never removed from the loop — only hidden from view. Behind every "autonomous" system are annotators, reviewers, feedback collectors, and engineers making judgment calls. HITL ML makes this visible and deliberate.

The human role is misunderstood. The human in the loop is not a temporary scaffold to be discarded when the model is good enough. Human judgment is the signal that defines what "good enough" means. You cannot specify the objective function, the reward, the label schema, or the evaluation metric without a human deciding what matters. The machine optimizes; the human decides what to optimize for.

You are misunderstood — and so am I. You are reading a book about being embedded in a system. While reading it, you are embedded in a system. The model that may have helped generate parts of this text was trained on human feedback. The annotations that trained the models you use were provided by humans whose names appear nowhere. We are all, in some sense, humans in the loop of something larger than ourselves.

This book does not pretend otherwise. It names these humans, describes their labor, and argues that understanding them is as important as understanding gradient descent.

What This Handbook Covers

Sixteen chapters across six parts, moving from foundations to frontiers:

Part I — Foundations. What HITL ML is, where it came from, and how to think about the space of human–machine interaction modes.

Part II — Core Techniques. Annotation and labeling, active learning, and interactive machine learning — the three classical pillars of HITL.

Part III — Learning from Human Feedback. Reinforcement learning from human feedback (RLHF), learning from demonstrations, and preference learning from comparisons and rankings — the paradigms powering modern AI alignment.

Part IV — Applications. NLP, computer vision, and healthcare — HITL through the lens of real domains with real constraints.

Part V — Systems and Practice. Annotation platforms, crowdsourcing quality control, and evaluation frameworks — the infrastructure that makes HITL work at scale.

Part VI — Ethics and Horizons. The humans behind the data: fairness, annotator welfare, bias, and where all of this is going.

A Note on Code

Every technical chapter includes runnable Python code. All examples are self-contained and use standard libraries: NumPy, scikit-learn, PyTorch, Hugging Face Transformers.

```python
# A taste of what's ahead: querying the most uncertain sample
import numpy as np
from sklearn.linear_model import LogisticRegression

rng = np.random.default_rng(42)
X = rng.normal(size=(100, 2))
y = (X[:, 0] + X[:, 1] > 0).astype(int)

model = LogisticRegression().fit(X[:20], y[:20])

probs = model.predict_proba(X[20:])
entropy = -np.sum(probs * np.log(probs + 1e-9), axis=1)
most_uncertain = np.argmax(entropy) + 20

print(f"Most uncertain sample index: {most_uncertain}")
print(f"Predicted probabilities:     {probs[most_uncertain - 20].round(3)}")
print()
print("The model doesn't know. So we ask a human.")
```

Notation

- $\mathcal{X}$ — input space; $\mathcal{Y}$ — label space
- $f_\theta : \mathcal{X} \to \mathcal{Y}$ — a model with parameters θ
- $\mathcal{U}$ — unlabeled pool; $\mathcal{L}$ — labeled dataset
- h — a human annotator; $\mathcal{H}$ — a set of annotators

You are a human in the loop. Let's begin.

Declaration of Originality

Submitted in partial non-fulfilment of the requirements for the degree of Doctor of Philosophy.

I hereby declare that the work presented in this handbook is my own, except where it is not.

The ideas in these pages were produced with the assistance of: researchers who published papers and occasionally put them behind paywalls; annotators whose labor made the models possible but whose names appear in no acknowledgments section; open-source contributors who wrote the libraries used in every code example; and the accumulated text of human civilization, ingested and compressed by large language models that have no opinion on any of it.

The errors are mine. The insights were collaborative.

I confirm that this work has not been submitted for any other degree or professional qualification, academic prize, grant, tenure case, or LinkedIn post, except as otherwise stated.

I further confirm that no SOTA (State of the Art) comparison section was included in this work. The reader will not find a table ranking 47 methods on 12 benchmarks, with the proposed method achieving statistically significant improvements on 9 of them ($\dagger p < 0.05$). This is not because such a table could not be constructed, but because its construction would be beside the point.

The point is this: the human is in the loop. Always has been. This book tries to understand what that means.

Signed: A Human in the Loop

Date: Whenever you are reading this

Institution: None. That's the point.

> **On the subtitle** :class: note
>
> *Misunderstood* refers to the field, to the human's role within it, and to the reader — all at once. If you think it refers only to you, you may be right. If you think it refers only to the machines, you are proving the subtitle's point.

Abstract

Human in the Loop Machine Learning: Misunderstood

The dominant narrative of machine learning positions the human as a temporary participant — present during data collection and model training, then progressively removed as automation improves. This handbook argues that this narrative is wrong, has always been wrong, and that understanding *why* it is wrong is one of the most practically useful things a machine learning practitioner can do.

Human-in-the-Loop (HITL) Machine Learning is the discipline of making human involvement in learning systems deliberate, structured, and ongoing. Rather than treating human feedback as a cost to be minimized, HITL treats it as a signal to be understood: where it comes from, what it encodes, how to collect it efficiently, and how to incorporate it faithfully into models that behave as their designers and users actually intend.

This handbook provides a comprehensive treatment of the field across six parts. Part I establishes foundations and a taxonomy of human–machine interaction. Part II covers the classical techniques: annotation and labeling, active learning, and interactive machine learning. Part III addresses the modern paradigms of alignment: reinforcement learning from human feedback (RLHF), learning from demonstrations, and preference learning. Part IV applies these techniques to natural language processing, computer vision, and healthcare. Part V covers the systems and infrastructure of HITL at scale: annotation platforms, crowdsourcing quality control, and evaluation methodology. Part VI examines the ethics and future of the field.

Each chapter includes formal definitions, mathematical treatment where appropriate, and self-contained runnable code examples in Python.

The handbook is intended for practitioners who build ML systems, researchers who study them, students who are learning about them, and anyone who has ever wondered who labeled the data that trained the model that made the decision that affected their life.

The answer, in all cases, is: a human. Often underpaid. Rarely credited. Always in the loop.

Keywords: human-in-the-loop machine learning, active learning, data annotation, reinforcement learning from human feedback, RLHF, preference learning, crowdsourcing, interactive machine learning, AI alignment, responsible AI

Subject classification: Machine Learning · Human-Computer Interaction · Artificial Intelligence · Data Science

Word count: Sufficient.

Acknowledgments

Every book about human-in-the-loop machine learning should begin with the humans in its own loop.

The annotators. This handbook is, in part, about you — about your labor, your judgment, and the fact that your name appears in no dataset card, paper acknowledgment, or press release about the models your work made possible. The field is beginning to reckon with this. It has a long way to go.

The open-source contributors whose libraries are used in every code example: the NumPy community (since 2006), the scikit-learn contributors (since 2007), the PyTorch team, the Hugging Face community, and the countless maintainers who fix bugs on weekends without compensation or recognition. Scientific computing is built on this invisible labor.

The researchers who published their work openly — in arXiv preprints, open-access venues, and GitHub repositories — making it possible for practitioners to learn without institutional access. And the researchers who reviewed papers carefully and honestly, a form of expert annotation that academia has somehow convinced itself is not labor.

The teachers who made knowledge accessible: the bloggers, the tutorial writers, the Stack Overflow answerers, the conference presenters who posted their slides. Learning does not happen in isolation, and it does not happen without people willing to explain things to strangers.

The critics. The researchers who pointed out that datasets encode biases, that annotator demographics shape model behavior, that the humans in the loop are often the most vulnerable participants in the system. This field is better for your insistence on asking uncomfortable questions.

There are known knowns — the things we know that we know. There are known unknowns — the things we know we do not know. There are unknown unknowns — the things we do not know we do not know.

And then there are the things that everyone in the field knows but that never quite make it into the papers.

This handbook is, in part, an attempt at the last category.

If you contributed to this field and do not see your name here: you are acknowledged. The loop includes you.

What Is Human-in-the-Loop Machine Learning?

> *A machine that can learn from experience… but only if you give it the right experiences. – paraphrasing Alan Turing*

The Automation Paradox

Every new wave of automation creates new demands on human attention. When airlines introduced autopilot, pilots became monitors — responsible not for moment-to-moment control but for the harder task of knowing *when* to take over. When supermarkets introduced self-checkout, they discovered that the systems require more human supervision per transaction than traditional cashiers, not less. And when machine learning began making decisions at scale — in medicine, finance, hiring, and content moderation — it created an enormous, ongoing demand for human judgment.

This is the **automation paradox** {cite}`bainbridge1983ironies`: the more capable an automated system becomes, the more consequential its failures are, and therefore the more necessary robust human oversight becomes. Machine learning is no exception.

Human-in-the-Loop Machine Learning (HITL ML) is the field that takes this paradox seriously and builds it into the design of systems from the start. Rather than treating human involvement as a temporary scaffolding to be eventually removed, HITL ML treats the human–machine interaction as a *feature* — a source of signal, correction, and guidance that can make models more accurate, more aligned with human values, and more trustworthy.

Defining Human-in-the-Loop ML

There is no single agreed-upon definition of HITL ML, and the term is used in several overlapping ways in the literature. For this handbook we adopt a broad but precise definition:

:::{admonition} Definition :class: important

Human-in-the-Loop Machine Learning refers to any ML system or methodology in which human feedback is incorporated into the learning process in a *deliberate, structured, and ongoing* manner — not just at the moment of dataset creation, but throughout training, evaluation, and deployment.

:::

This definition has three key clauses:

Deliberate. HITL is not accidental human influence (e.g., the fact that training data was originally created by humans). It refers to systems explicitly designed to solicit, incorporate, and benefit from human feedback.

Structured. The feedback has a defined form — a label, a correction, a preference judgment, a demonstration — and a defined role in the learning algorithm.

Ongoing. The feedback loop continues over time, allowing the system to improve as it encounters new situations, makes mistakes, and receives human guidance.

This definition includes classical annotation pipelines, active learning, interactive ML, RLHF, and imitation learning. It excludes passive data collection and purely offline supervised learning (though the line is blurry, as we will discuss in Chapter 2).

A Brief History

Expert Systems and Knowledge Engineering (1960s–1980s)

The earliest AI systems were almost entirely human-in-the-loop: knowledge engineers sat with domain experts for months, painstakingly encoding rules into expert systems like MYCIN and DENDRAL. Every piece of knowledge was explicitly provided by a human. The machine was the executor; the human was the oracle.

These systems worked surprisingly well in narrow domains but were brittle — unable to generalize beyond their hand-crafted rules and expensive to maintain.

The Statistical Turn (1980s–2000s)

The shift to statistical machine learning in the 1980s and 1990s changed the nature of human involvement. Instead of encoding knowledge as rules, humans now provided *examples* — labeled datasets that allowed models to infer patterns. The human role became that of annotator: labeling documents, tagging images, transcribing speech.

This was a major step forward, but it created a new bottleneck: **labeled data is expensive**. Researchers began asking how to make the most efficient use of human labeling effort. This question gave birth to **active learning**, first formalized in the early 1990s {cite}`cohn1994improving`.

The Deep Learning Era (2010s–present)

The deep learning revolution created a new regime: models with billions of parameters that can learn extraordinarily complex functions from data — but require commensurately enormous labeled datasets. ImageNet (14 million labeled images) and subsequent large-scale annotation projects showed both the power and the cost of scale.

At the same time, the deployment of ML at scale exposed new problems: models that were accurate on average but systematically wrong for specific groups, that hallucinated facts confidently, that optimized for measurable proxies rather than human values. These failures motivated new forms of human involvement: not just labeling, but *alignment* — the project of making models behave in the ways humans actually want.

The clearest expression of this alignment-focused HITL work is **Reinforcement Learning from Human Feedback (RLHF)** {cite}`christiano2017deep`, which became the backbone of systems like InstructGPT {cite}`ouyang2022training` and the instruction-following capabilities of modern language models.

Why HITL? The Case for Human Judgment

What makes human judgment valuable enough to incorporate into machine learning systems? Several properties:

1. Common Sense and World Knowledge

Humans bring vast background knowledge to any task. When a radiologist labels an X-ray, she is drawing on years of training, an understanding of anatomy, and implicit knowledge about what diseases look like — knowledge that is extraordinarily difficult to fully specify or acquire from data alone.

2. Semantic Grounding

Labels are meaningful because humans understand what they mean. The class "cat" in ImageNet refers to a fuzzy concept that humans recognize intuitively but which no formal definition fully captures. Models learn the label's extension (which images map to it) but may not learn the concept itself, leading to failures on edge cases that any human would handle correctly.

3. Value Alignment

Humans have preferences, values, and ethical judgments that ML models cannot derive from data alone. Whether a piece of text is "helpful" or "harmful" is not a purely empirical question — it depends on human values that vary across individuals and contexts. HITL is the primary mechanism by which these values can be communicated to ML systems.

4. Adaptability

Human judgment can adapt to novel situations without retraining. A model trained on historical data may fail catastrophically when the world changes; a human can recognize the novelty and respond appropriately.

5. Accountability

In high-stakes domains — medicine, law, finance — decisions need to be accountable to human beings. HITL systems make this accountability tractable by keeping humans in a position to understand, verify, and override machine behavior.

The Feedback Loop

The central structure of HITL ML is a feedback loop between a model and one or more humans:

```
+-----------------------------------------------+
|                                               |
|   Model makes predictions / requests         |
|   ----------------------------------->        |
|                                   Human      |
```

```
|   Human provides feedback        <-------- |
|   ----------------------------------        |
|                                             |
|   Model updates on feedback                |
|                                             |
+---------------------------------------------+
```

The nature of the feedback varies enormously across HITL paradigms:

Feedback type	Example	Primary paradigm
Class label	"This email is spam"	Supervised learning
Correction	"The entity should be ORG, not PER"	Active / interactive ML
Preference	"Response A is better than B"	RLHF / ranking
Demonstration	Showing the robot how to grasp an object	Imitation learning
Natural language	"Be more concise in your answers"	Instruction tuning
Implicit signal	User clicked / didn't click	Implicit feedback

---

What HITL Is Not

It is worth being precise about what falls *outside* our definition.

HITL is not the same as human-in-the-loop deployment (sometimes called "human on the loop"), where humans monitor automated decisions and can override them but are not feeding corrections back into training. We will discuss this distinction in Chapter 2.

HITL is not weak supervision alone. Programmatic labeling systems like Snorkel use labeling functions (often human-authored rules) to generate noisy labels at scale. This is a form of structured human input, but the feedback is not iterative in the way HITL typically implies.

HITL is not simply using labeled data. Every supervised learning model uses human-labeled data. HITL refers specifically to systems where the human feedback is an *active, iterative* part of the learning process.

The Economics of Human Feedback

Human feedback is valuable but costly. A medical imaging annotation can cost anywhere from tens to hundreds of dollars per image when performed by a specialist, depending on the subspecialty and task complexity {cite}`monarch2021human`. Crowdworker labels on platforms like Amazon Mechanical Turk can cost $0.01–$0.10 per item {cite}`hara2018data` at much lower quality. The fundamental challenge of HITL ML is **maximizing the value of each unit of human feedback**.

This leads to a unifying question that threads through most of this handbook:

:::{admonition} The Central Question of HITL ML :class: tip

Given a fixed budget of human time and attention, how should we decide what to ask humans, when to ask, and how to incorporate their answers into model training?

:::

The answer to this question depends on the domain, the form of feedback, the annotation cost, the risk of error, and the current state of the model — which is why HITL ML is a rich and still-evolving discipline.

Overview of the Handbook

The remainder of this handbook is structured as follows. **Part II** covers the three classical pillars of HITL: annotation, active learning, and interactive ML. **Part III** addresses the newer paradigms of learning from feedback — RLHF, imitation learning, and preference learning — that have become central to modern AI. **Part IV** examines HITL through the lens of specific application domains. **Part V** takes a practitioner perspective on platforms, crowdsourcing, and evaluation. **Part VI** addresses ethics and looks ahead.

> **See also:** For a practitioner-focused overview of the field, see {cite}`monarch2021human`. For the foundational active learning paper that kicked off much of the formal treatment of HITL, see {cite}`settles2009active`.

A Taxonomy of Human–Machine Interaction

A precise vocabulary is the prerequisite for clear thinking. The term "human-in-the-loop" is used loosely in practice — sometimes to mean that a human labels training data, sometimes that a human can override a model's decision, and sometimes that a human is actively directing the learning process in real time. These are meaningfully different things.

This chapter maps the landscape of human–machine interaction in ML, providing the conceptual vocabulary used throughout the rest of the handbook.

Levels of Automation

The most fundamental distinction is how actively the human is involved in the system's decision-making. A well-known framework from automation theory {cite}`sheridan1992telerobotics` distinguishes ten levels, but for ML purposes three categories capture the important variation:

Human-in-the-Loop (HITL)

The human is an *active participant in the learning process*. Decisions — or at least, consequential decisions — require human input before they are finalized. The system cannot operate without ongoing human engagement.

Examples: An active learning system that queries a clinician before adding a case to training data. A data annotator labeling examples that are immediately used for model updates. An RLHF labeler comparing model outputs.

Human-on-the-Loop (HOTL)

The system operates autonomously but a human monitors it and can intervene. The human is a *supervisor*, not a participant. Feedback may or may not flow back into training.

Examples: A content moderation system that flags posts automatically; a human reviewer samples and corrects a fraction of decisions. An autopilot that flies the plane but alerts the pilot to anomalies.

Human-in-Command (HIC)

The human makes all decisions; the system provides *recommendations or information* but has no autonomy. This is the weakest form of ML deployment.

Examples: A diagnostic support system that shows a doctor the model's probability estimate but leaves the final decision entirely to the physician.

> **Which level is right?** :class: tip
>
> The appropriate level of automation depends on the cost of errors, the reliability of the model, the expertise of available humans, and the latency constraints of the task. These factors change as a model matures — most systems start HITL and migrate toward HOTL as confidence grows.

```
Raw data --> Preprocessing --> Features --> Training --> Evaluation --> Deployment
    ^              ^                            ^              ^              ^
    |              |                            |              |              |
 Collection    Annotation                   Active          Testing       Monitoring
 feedback      & labeling                  learning        feedback      & correction
```

Stage	Human role	Chapter
Collection	Deciding what data to collect; sampling strategy	3, 4
Annotation	Assigning labels, structures, metadata	3, 13
Training	Active learning queries; online feedback	4, 5, 6
Evaluation	Human evaluation of model outputs	14
Deployment	Monitoring, exception handling, corrections	12, 14

Active vs. Passive Human Involvement

In *active* HITL, the system selects which data points to present to the human — asking questions strategically. In *passive* HITL, the human provides feedback on whatever data happens to arrive (e.g., data labeling batches assigned sequentially).

Active involvement is more efficient because feedback is directed where it will most improve the model. Passive involvement is simpler to implement and manage.

A related distinction is between **batch** and **online** feedback:

- **Batch:** Humans label a large pool of examples; the model retrains. Repeat.
- **Online (streaming):** Human feedback arrives continuously; the model updates incrementally.

Batch workflows are the norm in industry (annotation projects followed by training runs). Online workflows are more natural for interactive systems and reinforcement learning settings.

Single vs. Multiple Annotators

Most formal presentations of HITL assume a single, consistent annotator. In practice, annotation involves many people, and their judgments differ.

Aggregation combines multiple annotations into a single label, typically by majority vote or a statistical model (Chapter 13).

Disagreement as signal — some researchers argue that annotator disagreement is valuable information that should not be collapsed into a single "gold" label. Perspectivist approaches to

NLP, for instance, preserve multiple annotations as soft labels that reflect the genuine ambiguity of the data {cite}`uma2021learning`.

A Unified Framework

We can represent any HITL configuration with a five-tuple:

$$\text{HITL config} = (\text{level}, \text{modality}, \text{stage}, \text{frequency}, \text{annotators})$$

where:

- **level** $\in$ {HITL, HOTL, HIC}
- **modality** $\in$ {label, correction, demonstration, preference, NL, implicit}
- **stage** $\in$ {collection, annotation, training, evaluation, deployment}
- **frequency** $\in$ {batch, online}
- **annotators** $\in \mathbb{N}^+$ (number of annotators per item)

This taxonomy lets us compare diverse HITL systems on the same axes and reason about the trade-offs between them. The rest of the handbook drills into specific cells of this space.

> **See also:** For an empirical study of how annotation decisions affect downstream model behavior, see {cite}`bender2021stochastic`. For a survey of interactive ML systems, see {cite}`amershi2014power`.

Data Annotation and Labeling

Data annotation is the most ubiquitous form of human involvement in ML. Before a model can learn, someone must tell it what the correct answers are — and that someone is usually a human. This chapter covers the theory and practice of annotation: what makes annotation hard, how to design annotation tasks, how to measure quality, and how to handle disagreement.

Types of Annotation

Annotation tasks vary enormously in their structure, difficulty, and cost. The major types include:

Classification

The annotator assigns each instance to one of K predefined categories. This is the simplest annotation task cognitively, but defining a good category scheme (a *taxonomy*) can be surprisingly difficult.

Binary classification (is this image a cat?) is the simplest case. **Multiclass** (what species is this animal?) requires annotators to choose one option from a list. **Multilabel** annotation (what topics does this article cover?) allows multiple simultaneous labels.

Sequence Labeling

Each token in a sequence receives a label. Named Entity Recognition (NER) is the canonical example — annotators mark spans of text as PERSON, ORGANIZATION, LOCATION, etc. Annotation is typically performed using the BIO (Beginning-Inside-Outside) or BIOES tagging scheme:

```
B-ORG    O            B-ORG    O      O      O
```

Span and Relation Annotation

Beyond labeling individual tokens, annotators may need to: - Identify spans (multi-token expressions) and assign types - Mark *relations* between spans ("Apple" ACQUIRED "Shazam") - Annotate coreference chains (all mentions of the same entity)

These tasks are cognitively demanding and have lower inter-annotator agreement.

Bounding Boxes and Object Detection

Annotators draw rectangles around objects in images and assign a category label to each box. Localization precision matters: a box that is too small misses context; a box that is too large

includes background. Modern annotation tools compute intersection-over-union (IoU) with reference annotations to flag quality issues.

Segmentation

Pixel-level annotation: each pixel is assigned to a class (semantic segmentation) or to a specific object instance (instance segmentation). High quality segmentation is among the most expensive annotation types, with costs ranging from tens to over a hundred dollars per image for complex scenes depending on domain and tool support.

Transcription and Translation

Audio □ text (ASR training data), handwriting □ text (OCR data), or source language □ target language (MT data). These tasks require linguistic expertise and cannot be done reliably by untrained annotators.

Annotation Guidelines

The single most important determinant of annotation quality is the quality of the **annotation guidelines**: the written instructions annotators follow.

Good guidelines: - State the task goal and explain *why* the label matters - Provide a clear definition for each category with positive and negative examples - Address common edge cases and hard cases explicitly - Specify what to do when uncertain (e.g., mark "skip" vs. forced choice) - Include worked examples of complete annotation

Poor guidelines rely on annotators to "use common sense" for edge cases — which leads to inconsistent decisions that degrade model quality and inflate inter-annotator disagreement.

> **Guideline development is iterative** :class: note
>
> Do not expect to write perfect guidelines upfront. Run a small pilot annotation round, analyze disagreements, and update the guidelines. Repeat. Well-developed guidelines typically go through 3–5 revision cycles before they stabilize.

Measuring Annotation Quality: Inter-Annotator Agreement

When multiple annotators label the same data, their agreement can be measured. High agreement suggests the task is well-defined and the annotators understood it. Low agreement suggests ambiguity in the task, the guidelines, or the data itself.

Cohen's Kappa

For two annotators labeling data into K categories, **Cohen's kappa** {cite}`cohen1960coefficient` corrects observed agreement for chance:

$$\kappa = \frac{P_o - P_e}{1 - P_e}$$

where P_o is observed proportional agreement and P_e is the probability of agreement by chance (computed from the marginal label distributions).

$\kappa = 1$ means perfect agreement; $\kappa = 0$ means agreement no better than chance; $\kappa < 0$ means systematic disagreement.

κ range	Interpretation
< 0	Less than chance
$0.0 - 0.20$	Slight
$0.21 - 0.40$	Fair
$0.41 - 0.60$	Moderate
$0.61 - 0.80$	Substantial
$0.81 - 1.00$	Almost perfect

Fleiss' Kappa

Extends Cohen's kappa to $M > 2$ annotators. Each annotator independently labels each item; the formula aggregates across annotators:

$$\kappa_F = \frac{\bar{P} - \bar{P}_e}{1 - \bar{P}_e}$$

where $\bar{P}$ is the mean pairwise agreement across all annotator pairs, and $\bar{P}_e$ is the expected agreement under random assignment.

Krippendorff's Alpha

The most general metric, supporting any number of annotators, any scale type (nominal, ordinal, interval, ratio), and missing data {cite}`krippendorff2011computing`:

$$\alpha = 1 - \frac{D_o}{D_e}$$

where D_o is observed disagreement and D_e is expected disagreement. Krippendorff's alpha is generally preferred in academic work because of its flexibility.

```
import numpy as np
from sklearn.metrics import cohen_kappa_score

# Simulate two annotators labeling 200 items into 3 categories
rng = np.random.default_rng(0)
true_labels = rng.integers(0, 3, size=200)

# Annotator 1: mostly agrees with ground truth
ann1 = true_labels.copy()
flip_mask = rng.random(200) < 0.15
ann1[flip_mask] = rng.integers(0, 3, size=flip_mask.sum())

# Annotator 2: less consistent
ann2 = true_labels.copy()
flip_mask2 = rng.random(200) < 0.30
```

```
ann2[flip_mask2] = rng.integers(0, 3, size=flip_mask2.sum())

kappa_12 = cohen_kappa_score(ann1, ann2)
kappa_1true = cohen_kappa_score(ann1, true_labels)
kappa_2true = cohen_kappa_score(ann2, true_labels)

print(f"Cohen's κ (ann1 vs ann2):   {kappa_12:.3f}")
print(f"Cohen's κ (ann1 vs truth):  {kappa_1true:.3f}")
print(f"Cohen's κ (ann2 vs truth):  {kappa_2true:.3f}")
```

Handling Disagreements

When annotators disagree, there are several strategies:

Majority Vote

The most common label is taken as the gold standard. Simple and robust when the number of annotators per item is odd. Fails when a minority group of annotators is systematically more accurate.

Weighted Voting

Annotators are weighted by their estimated accuracy (derived from agreement with a gold standard or other annotators). More accurate annotators have more influence.

Soft Labels

Rather than collapsing annotations to a single label, preserve the distribution. If 3 of 5 annotators said "positive" and 2 said "neutral," represent this as $(p_{\text{pos}}, p_{\text{neutral}}, p_{\text{neg}}) = (0.6, 0.4, 0.0)$. Training on soft labels improves calibration.

Arbitration

A senior annotator or domain expert adjudicates disagreements. Gold standard but expensive; typically reserved for high-stakes domains.

Statistical Models

More sophisticated approaches model annotator competence probabilistically. The **Dawid-Skene** model {cite}`dawid1979maximum` simultaneously estimates annotator confusion matrices and item true labels via EM. See Chapter 13 for details.

Label Noise and Its Effects

Real annotation is noisy. The effects of label noise on model training depend on the noise type:

- **Random noise** (labels randomly flipped) degrades performance but models are surprisingly robust to moderate levels (up to ~20% for many tasks).

- **Systematic/adversarial noise** (labels consistently wrong in specific patterns) is far more damaging and harder to detect.
- **Class-conditional noise** (errors more likely for certain classes) biases the model's decision boundary.

A practical rule of thumb: with n training examples and ϵ fraction of corrupted labels, model performance degrades roughly as if you had $(1-2\epsilon)^2 n$ clean examples {cite}`natarajan2013learning`. For $\epsilon = 0.2$, this is equivalent to losing 36% of your data.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

rng = np.random.default_rng(42)
X, y = make_classification(n_samples=2000, n_features=20, random_state=42)

noise_levels = np.linspace(0, 0.45, 15)
mean_accs = []

for eps in noise_levels:
    y_noisy = y.copy()
    flip = rng.random(len(y)) < eps
    y_noisy[flip] = 1 - y_noisy[flip]
    scores = cross_val_score(LogisticRegression(max_iter=500), X, y_noisy, cv=5)
    mean_accs.append(scores.mean())

plt.figure(figsize=(7, 4))
plt.plot(noise_levels, mean_accs, 'o-', color='#2b3a8f', linewidth=2)
plt.xlabel("Label noise rate (ε)", fontsize=12)
plt.ylabel("Cross-validated accuracy", fontsize=12)
plt.title("Effect of Label Noise on Model Performance", fontsize=13)
plt.axvline(0.2, color='#e05c5c', linestyle='--', alpha=0.7, label='20% noise')
plt.legend()
plt.tight_layout()
plt.savefig('label_noise_effect.png', dpi=150)
plt.show()
print(f"\nAccuracy at 0% noise:  {mean_accs[0]:.3f}")
print(f"Accuracy at 20% noise: {mean_accs[round(0.2 / 0.45 * 14)]:.3f}")
print(f"Accuracy at 40% noise: {mean_accs[-2]:.3f}")
```

Annotation Cost and Throughput

Understanding annotation economics is essential for project planning.

Task type	Typical throughput	Cost per item (specialist)
Binary image classification	200–500/hr	\$0.02–0.10
NER (short text)	50–150 items/hr	\$0.10–0.50
Relation extraction	20–60 items/hr	\$0.30–1.50

Task type	Typical throughput	Cost per item (specialist)
Medical image segmentation	5–30 items/hr	$10–100
Video annotation	5–20 min video/hr	$20–200

These figures are approximate order-of-magnitude estimates and vary enormously by domain expertise required, annotation tool quality, guideline clarity, and annotator experience. They should be treated as illustrative, not prescriptive.

> **See also:** Annotation tool options are covered in Chapter 12. Statistical models for crowdsourced annotation (Dawid-Skene, MACE) are covered in Chapter 13.

Active Learning

Labeled data is expensive. The core insight of active learning is that *not all unlabeled examples are equally informative* — a model can improve faster if it gets to choose which examples to ask about. Rather than labeling data randomly, an active learning system queries an oracle (usually a human annotator) on the examples most likely to improve the model.

This chapter covers the theory and practice of active learning: query strategies, sampling frameworks, stopping criteria, and practical considerations for real deployments.

The Active Learning Setup

The standard **pool-based active learning** setting involves:

- A **labeled set** $\mathcal{L} = \{(x_i, y_i)\}_{i=1}^n$ — initially small
- An **unlabeled pool** $\mathcal{U} = \{x_j\}_{j=1}^m$ — typically much larger than $\mathcal{L}$
- An **oracle** $\mathcal{O}$ that can return $y = \mathcal{O}(x)$ for any queried x
- A **query strategy** ϕ that selects the next query $x^* = \phi(\mathcal{L}, \mathcal{U}, f_\theta)$

The active learning loop:

```
1. Initialize: L = small labeled seed set, U = unlabeled pool
2. Train: f_θ ← train(L)
3. Query: x* = argmax φ(x; f_θ) over x ∈ U
4. Label: y* = O(x*)
5. Update: L ← L ∪ {(x*, y*)}, U ← U \ {x*}
→ Repeat from 2 until budget exhausted
```

The goal is to reach a target model quality using as few oracle queries as possible.

Theoretical Foundations

A natural question is: how much can active learning help? In the best case, active learning can achieve *exponential* reductions in label complexity — reaching error ϵ with $O(\log(1/\epsilon))$ labels rather than the $O(1/\epsilon)$ needed by passive learning, at least in realizable settings with a good query strategy {cite}`settles2009active`.

In practice, guarantees are harder to obtain. **Agnostic active learning** {cite}`balcan2006agnostic` shows that label savings are possible even when the target concept is not in the hypothesis class, but the savings depend strongly on the disagreement coefficient — a measure of how quickly the set of plausible hypotheses shrinks as data accumulates.

The key practical implication: active learning's advantage is largest when the **decision boundary is simple and concentrated** (so uncertainty queries quickly eliminate wrong hypotheses), and smallest when the hypothesis class is large or the boundary is complex.

Query Strategies

Uncertainty Sampling

The simplest and most widely used strategy: query the example on which the model is *most uncertain* {cite}`lewis1994sequential`.

Least confidence queries the example for which the model is least confident in its top prediction:

$$x^* = \arg\max_{x \in \mathcal{U}} (1 - P_\theta(\hat{y} \mid x))$$

Margin sampling considers the gap between the top two predicted probabilities:

$$x^* = \arg\min_{x \in \mathcal{U}} (P_\theta(\hat{y}_1 \mid x) - P_\theta(\hat{y}_2 \mid x))$$

Entropy sampling uses the full predicted distribution:

$$x^* = \arg\max_{x \in \mathcal{U}} \left(-\sum_{k=1}^{K} P_\theta(y_k \mid x) \log P_\theta(y_k \mid x) \right)$$

Entropy sampling is the most principled of the three — it considers all classes — and generally outperforms the others on multiclass problems.

Query by Committee (QbC)

Train a **committee** of C models (using bagging, different initializations, or different architectures). Query the example on which the committee disagrees most:

$$x^* = \arg\max_{x \in \mathcal{U}} \text{ disagreement}(\{f_c(x)\}_{c=1}^{C})$$

Disagreement can be measured as **vote entropy** (entropy over the committee's majority votes) or **KL divergence** from the consensus distribution.

QbC provides better uncertainty estimates than a single model but requires training multiple models, which is computationally expensive.

Expected Model Change

Query the example that would cause the greatest change to the current model if labeled. For gradient-based models, this corresponds to the example with the greatest expected gradient magnitude {cite}`settles2008analysis`:

$$x^* = \arg\max_{x \in \mathcal{U}} \sum_{y \in \mathcal{Y}} P_\theta(y \mid x) \, \|\nabla_\theta \mathcal{L}(f_\theta(x), y)\|$$

This strategy has strong theoretical motivation but requires computing gradients for each candidate, making it expensive for large models.

Core-Set / Geometric Approaches

Uncertainty-based strategies can be **biased toward outliers**: an unusual example may be highly uncertain but not representative of the data distribution. Core-set methods address this by seeking a diverse sample that covers the feature space.

The **k-center greedy** algorithm {cite}`sener2018active` finds the smallest set of points such that every unlabeled point is within δ of at least one queried point:

$$x^* = \arg\max_{x \in \mathcal{U}} \min_{x' \in \mathcal{L}} d(x, x')$$

i.e., query the point farthest from any currently labeled point. This encourages a well-spread set of annotations.

BADGE

Batch Active learning by Diverse Gradient Embeddings {cite}`ash2020deep` combines uncertainty and diversity: it selects a batch of examples whose gradient embeddings (with respect to the predicted label) are both large in magnitude (uncertain) and diverse (covering different regions of the gradient space). This is one of the most competitive modern strategies.

Uncertainty Estimation for Deep Models

The strategies above assume access to calibrated probability outputs from the model. For simple models (logistic regression, softmax classifiers), this is straightforward. For deep networks, getting reliable uncertainty estimates requires additional technique.

Two Kinds of Uncertainty

Following Kendall and Gal {cite}`kendall2017uncertainties`, we distinguish:

Aleatoric uncertainty (data uncertainty): inherent noise in the observations that cannot be reduced by collecting more data. A blurry image is aleatorically uncertain — no amount of additional training data from the same distribution will make the model more confident on it.

Epistemic uncertainty (model uncertainty): uncertainty due to limited training data or a model that has not seen similar examples. Epistemic uncertainty *can* be reduced by labeling more data — and is therefore the relevant quantity for active learning query selection.

For active learning, we want to query examples with high epistemic uncertainty, not high aleatoric uncertainty. Querying a fundamentally ambiguous example wastes the oracle's effort: no label they provide will be clearly correct.

Monte Carlo Dropout

A practical approach to epistemic uncertainty estimation for neural networks is **MC Dropout** {cite}`gal2016dropout`: apply dropout at inference time and run T forward passes. The variance across predictions is an estimate of epistemic uncertainty.

```
import numpy as np
import torch
import torch.nn as nn

torch.manual_seed(42)
rng = np.random.default_rng(42)

class MCDropoutNet(nn.Module):
    def __init__(self, input_dim=20, hidden=64, output_dim=2, p_drop=0.3):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden), nn.ReLU(), nn.Dropout(p_drop),
            nn.Linear(hidden, hidden),    nn.ReLU(), nn.Dropout(p_drop),
            nn.Linear(hidden, output_dim)
        )

    def forward(self, x):
        return self.net(x)

def mc_uncertainty(model, x, T=30):
    """
    Run T stochastic forward passes with dropout active.
    Returns mean prediction and epistemic uncertainty (predictive variance).
    """
    model.train()  # keep dropout active during inference
    with torch.no_grad():
        preds = torch.stack([
            torch.softmax(model(x), dim=-1) for _ in range(T)
        ])  # shape: (T, N, C)
    mean_pred = preds.mean(0)
    # Epistemic uncertainty: mean of variances across passes
    epistemic = preds.var(0).sum(-1)
    # Aleatoric uncertainty: entropy of mean prediction
    aleatoric = -(mean_pred * (mean_pred + 1e-9).log()).sum(-1)
    return mean_pred, epistemic, aleatoric

# Quick demonstration
model = MCDropoutNet(input_dim=20, output_dim=2)
# In-distribution example (simulated)
x_familiar   = torch.randn(1, 20) * 0.5
# Out-of-distribution example (far from training distribution)
x_unfamiliar = torch.randn(1, 20) * 3.0

for name, x in [("In-distribution ", x_familiar), ("Out-of-distribution", x_unfamiliar)]:
    _, ep, al = mc_uncertainty(model, x)
    print(f"{name} | epistemic: {ep.item():.4f} | aleatoric: {al.item():.4f}")
```

In the untrained network above, both examples show similar uncertainty. After training, the out-of-distribution example will show higher epistemic uncertainty — the model has not learned a reliable mapping for inputs far from the training distribution.

Deep Ensembles

Training M independently initialized models and averaging their predictions provides a simpler and often more reliable uncertainty estimate than MC Dropout {cite}`lakshminarayanan2017simple`. The disagreement between ensemble members is the epistemic uncertainty signal.

For active learning at scale, both MC Dropout and deep ensembles add overhead proportional to T or M forward passes. In practice, $T = 10$–30 for MC Dropout or $M = 5$ ensemble members is often sufficient to rank examples by epistemic uncertainty, even if the absolute values are not well-calibrated.

A Full Active Learning Loop

```python
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from copy import deepcopy

rng = np.random.default_rng(42)

# Generate dataset
X, y = make_classification(
    n_samples=2000, n_features=20, n_informative=10,
    n_classes=3, n_clusters_per_class=1, random_state=42
)
X_train, y_train = X[:1500], y[:1500]
X_test,  y_test  = X[1500:], y[1500:]

def entropy_query(model, X_pool):
    """Return index of most uncertain sample (entropy)."""
    probs = model.predict_proba(X_pool)
    ent = -np.sum(probs * np.log(probs + 1e-9), axis=1)
    return np.argmax(ent)

def random_query(X_pool):
    """Random baseline."""
    return rng.integers(0, len(X_pool))

def run_active_learning(strategy='entropy', n_initial=30, n_queries=120, query_batch=5):
    labeled_idx = list(rng.choice(len(X_train), n_initial, replace=False))
    unlabeled_idx = [i for i in range(len(X_train)) if i not in labeled_idx]
    accs = []

    for step in range(n_queries // query_batch):
```

```
        model = LogisticRegression(max_iter=500, C=1.0)
        model.fit(X_train[labeled_idx], y_train[labeled_idx])
        accs.append(accuracy_score(y_test, model.predict(X_test)))

        # Query
        X_pool = X_train[unlabeled_idx]
        for _ in range(query_batch):
            if strategy == 'entropy':
                q = entropy_query(model, X_pool)
            else:
                q = random_query(X_pool)
            labeled_idx.append(unlabeled_idx.pop(q))
            X_pool = X_train[unlabeled_idx]

    return np.array(accs)

labels_used = np.arange(1, 25) * 5 + 30  # label counts at each step

accs_active = run_active_learning(strategy='entropy')
accs_random = run_active_learning(strategy='random')

plt.figure(figsize=(8, 5))
plt.plot(labels_used, accs_active, 'o-', label='Entropy sampling', color='#2b3a8f', linewidth
plt.plot(labels_used, accs_random, 's--', label='Random baseline',  color='#e05c5c', linewidt
plt.xlabel("Number of labeled examples", fontsize=12)
plt.ylabel("Test accuracy", fontsize=12)
plt.title("Active Learning vs. Random Sampling", fontsize=13)
plt.legend(fontsize=11)
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('active_learning_curve.png', dpi=150)
plt.show()

print(f"Active learning reaches {accs_active[-1]:.3f} accuracy")
print(f"Random sampling reaches {accs_random[-1]:.3f} accuracy")
print(f"Active learning saves ~{int((accs_random.tolist().index(min(accs_random, key=lambda a
```

The Cold Start Problem

Active learning requires a trained model to score unlabeled points — but at the beginning, you have no (or very few) labeled examples. This is the **cold start problem**.

Practical solutions:

1. **Random initialization:** Label a small random seed set (20–100 examples) before starting active learning.
2. **Clustering-based initialization:** Use k-means on the unlabeled pool; label one example from each cluster. This ensures diversity in the initial labeled set.
3. **Embedding-based selection:** Use a pre-trained encoder to embed examples; select a diverse subset via core-set.

For most tasks, a few dozen random seed labels are typically sufficient to get active learning started; the exact number depends on class balance, feature dimensionality, and model complexity.

Batch-Mode Active Learning

In practice, annotators work in batches — it is inefficient to train and deploy a new model after each single label. **Batch-mode active learning** selects a set of b examples to label simultaneously.

Naively selecting the top-b most uncertain examples leads to **redundancy**: highly uncertain examples tend to cluster together (e.g., examples near the decision boundary in the same region). Better batch strategies optimize for both uncertainty *and* diversity within the batch.

Determinantal Point Processes (DPPs) provide a principled way to sample diverse batches: they define a distribution over subsets that penalizes similar items. The probability of a subset S under a DPP is proportional to $\det(L_S)$ where L is a kernel matrix encoding similarity.

Stopping Criteria

When should active learning stop? Common criteria:

- **Budget exhausted:** Simplest — stop when the annotation budget runs out.
- **Performance plateau:** Stop when model accuracy on a held-out validation set has not improved by more than δ for k consecutive rounds.
- **Confidence threshold:** Stop when fewer than some fraction of unlabeled examples have uncertainty above a threshold.
- **Maximum loss reduction:** Estimate the maximum possible gain from additional labels; stop when this falls below a threshold {cite}`bloodgood2009method`.

When Active Learning Works (and When It Doesn't)

Active learning tends to work well when: - Labeling is expensive and the unlabeled pool is large - The data has clear structure the model can exploit to identify informative examples - The model class is appropriate for the task

Active learning performs poorly when: - The initial model is very poor (cold start) and cannot meaningfully rank examples - The query strategy selects outliers or mislabeled examples (noise robustness matters) - The data distribution shifts between the unlabeled pool and the test distribution

A key practical concern is **distribution mismatch**: active learning tends to query examples near the decision boundary, creating a biased labeled set that may not represent the test distribution well. This can lead to well-trained decision boundaries but poor calibration.

> **See also:** The foundational survey is {cite}`settles2009active`. Theoretical foundations (label complexity, agnostic bounds): {cite}`balcan2006agnostic`. For deep learning-specific active learning, see {cite}`ash2020deep` (BADGE) and {cite}`sener2018active` (core-set). For a critical evaluation of when active

learning actually helps, see {cite}`lowell2019practical`. On aleatoric vs. epistemic uncertainty for deep models, see {cite}`kendall2017uncertainties`; for deep ensembles as uncertainty estimators, see {cite}`lakshminarayanan2017simple`; for MC Dropout as approximate Bayesian inference, see {cite}`gal2016dropout`.

Interactive Machine Learning

Active learning asks a focused question: given a budget, which examples should I label? Interactive Machine Learning (IML) asks a broader question: how can we design the *entire interaction* between a human and a learning system to be maximally productive, enjoyable, and correct?

IML is distinguished by the **immediacy** and **directness** of the human–model feedback loop. Where traditional ML involves a human handing off data and waiting for training to complete, IML enables humans to observe model behavior, provide feedback, and see the model respond — often within seconds.

Principles of Interactive Machine Learning

Amershi et al. {cite}`amershi2014power` identify three defining characteristics of IML:

1. Rapid Feedback: The model updates quickly enough for humans to perceive the effect of their feedback. In the limit, model updates happen in real time.

2. Direct Manipulation: The human interacts with the model through the data or through the model's predictions — not through configuration files or hyperparameter tuning.

3. Iterative Refinement: The process is genuinely iterative: the human's next action is informed by the model's current behavior, which was shaped by the human's previous actions.

This creates a tight **co-adaptive loop**: both the human and the model change over time in response to each other. The human learns what the model understands; the model learns what the human cares about.

Comparison with Active Learning

IML and active learning overlap considerably but are not identical:

Property	Active Learning	Interactive ML
Primary question	What to label?	How to interact?
Feedback latency	Can be batch (days)	Typically real-time or near-real
Model update frequency	Per round (batch)	Per interaction (online)
Human agency	Answers model's questions	Can proactively teach model
Interface design	Secondary concern	Central concern
Cognitive load of human	Not explicitly modeled	Explicitly considered

In active learning, the machine directs the interaction. In IML, the human can also take the initiative — providing examples, corrections, or feedback on whatever aspect of model behavior seems most problematic.

Mixed-Initiative Interaction

Mixed-initiative systems allow both the human and the machine to take the lead at different moments {cite}`allen1999mixed`. A purely machine-directed system asks questions and the human answers. A purely human-directed system lets the human decide everything. Mixed-initiative systems balance both.

In practice, the best IML systems combine: - **Machine initiative:** "I'm unsure about these examples — can you label them?" - **Human initiative:** "I notice the model is consistently wrong about this category — let me provide more examples" - **Confirmation:** The model surfaces its current understanding; the human confirms or corrects

Good IML interfaces make the model's current understanding visible and correctable. This is the **intelligibility** requirement: humans can only guide a model they understand, at least approximately.

Human Factors in IML

IML brings human factors — cognitive load, fatigue, consistency, and trust — directly into the learning loop. Poor IML design leads to:

Annotation fatigue: Humans making faster, sloppier decisions as sessions lengthen. Errors that enter the training data.

Anchoring bias: Humans over-relying on the model's current suggestions. If an interface pre-fills the model's prediction, annotators are less likely to correct it even when it is wrong — a systematic source of label noise that compounds across annotation rounds {cite}`geva2019annotator`. Pre-annotation can speed throughput {cite}`lingren2014evaluating` while simultaneously reducing the rate at which annotators catch model errors; these two effects must be weighed against each other in IML interface design.

Trust miscalibration: Humans either over-trust (accepting wrong model outputs) or under-trust (ignoring correct suggestions). Both patterns reduce the value of the human–model collaboration.

Session consistency: Humans may make different decisions on the same example at different times, especially after long sessions. Consistency checks (re-presenting earlier examples) can detect and correct this.

Good IML design mitigates these issues through interface choices: presenting model confidence explicitly, randomizing display order, limiting session length, and building in consistency checks.

The IML Feedback Types in Practice

Example-Level Feedback

The human provides a label or correction for a specific example. This is the most common form and is directly compatible with supervised learning.

Feature-Level Feedback

The human indicates which features are relevant or irrelevant. "The model should pay attention to the words 'urgent' and 'deadline' for this category." This is more expressive than example-level feedback and can be more efficient for certain tasks.

TFIDF Interactive and similar systems allow annotators to highlight relevant words in text documents. These highlights are converted into constraints or additional supervision on the model's attention.

Model-Level Feedback

The human directly corrects the model's behavior on a class of inputs: "Whenever the input contains [X], the output should be [Y]." This maps to logic rules or constraints in approaches like Posterior Regularization {cite}`ganchev2010posterior` or constraint-driven learning.

Case Study: Google Teachable Machine

Teachable Machine is an accessible web-based IML system that lets non-technical users train image classifiers in the browser. The user:

1. Records examples of each class using their webcam
2. Trains the model with a single click (fine-tuning a MobileNet in the browser)
3. Immediately sees the model's predictions on live video
4. Adds more examples for classes where the model is wrong

This illustrates the core IML loop: provide examples □ observe model □ identify failure □ provide more targeted examples. The real-time feedback (model outputs update in real time, typically at interactive frame rates on modern hardware) makes the co-adaptation loop viscerally immediate.

Implementing a Simple IML Loop

```
import numpy as np
from sklearn.linear_model import SGDClassifier
from sklearn.preprocessing import StandardScaler
import warnings
warnings.filterwarnings('ignore')

class SimpleIMLSystem:
    """
    Minimal IML system that allows online feedback and displays
    model state after each annotation.
```

```
    """

    def __init__(self, n_features=10, n_classes=2):
        self.model = SGDClassifier(loss='log_loss', max_iter=1, warm_start=True,
                                   random_state=42)
        self.scaler = StandardScaler()
        self.X_seen = []
        self.y_seen = []
        self.n_classes = n_classes
        self.initialized = False

    def update(self, x, y_true):
        """Receive a single labeled example and update the model."""
        self.X_seen.append(x)
        self.y_seen.append(y_true)

        if len(self.X_seen) >= 2 * self.n_classes:
            X_arr = np.array(self.X_seen)
            y_arr = np.array(self.y_seen)
            X_scaled = self.scaler.fit_transform(X_arr)
            self.model.partial_fit(X_scaled[-1:], y_arr[-1:],
                                   classes=list(range(self.n_classes)))
            self.initialized = True

        return self

    def predict_with_confidence(self, x):
        """Predict label and return confidence."""
        if not self.initialized:
            return None, 0.0
        x_scaled = self.scaler.transform(x.reshape(1, -1))
        probs = self.model.predict_proba(x_scaled)[0]
        return self.model.predict(x_scaled)[0], probs.max()

    def current_accuracy(self, X_val, y_val):
        if not self.initialized:
            return None
        X_scaled = self.scaler.transform(X_val)
        return (self.model.predict(X_scaled) == y_val).mean()


# Simulate an IML session
rng = np.random.default_rng(42)
X_all, y_all = np.random.default_rng(0).random((500, 10)), np.random.default_rng(0).integers(
X_val, y_val = X_all[400:], y_all[400:]

system = SimpleIMLSystem(n_features=10, n_classes=2)
accs, confidences = [], []

print("Step | Labels | Accuracy | Example confidence")
print("-" * 50)
```

```
for step in range(100):
    x, y = X_all[step], y_all[step]
    pred, conf = system.predict_with_confidence(x)
    system.update(x, y)

    if (step + 1) % 10 == 0:
        acc = system.current_accuracy(X_val, y_val)
        if acc is not None:
            accs.append(acc)
            print(f"  {step+1:3d} |  {step+1:4d}  |  {acc:.3f}   | {conf:.3f}")
```

The Grandmother Test

One useful heuristic for evaluating IML interface design — and HITL system design more broadly — is what we will call the **Grandmother Test** (an original coinage introduced here as a design constraint, not as a reference to prior work):

> *A woman born in 1930 should be able to use this device by voice, and if frustrated, gracefully fall back to keyboard or text interface.*

The test is not primarily about accessibility, though it is that too. It is about **designing for friction**. If a system requires a mental model of neural networks, training loops, or probability distributions to use effectively, it has failed the Grandmother Test. The human in the loop should be able to participate without understanding the machine side of the loop.

The implications for IML design are concrete:

Voice-first fallback: The primary interaction modality should be natural language or gesture — not parameter sliders or confidence thresholds. Experts may want sliders; everyone should be able to say "that's wrong."

Graceful degradation: When the user's preferred modality fails or frustrates, the system should offer an alternative — not a blank screen or an error message. The interface is part of the learning system; a user who cannot interact cannot teach.

Legible model state: The model's current understanding should be visible in human terms. Not "confidence: 0.73" but "I'm fairly sure this is [X], but I've seen examples like this go both ways." Uncertainty should be communicated in language that invites correction.

Tolerance for ambiguity: A 93-year-old user and a 23-year-old ML engineer will interact differently with the same system. The Grandmother Test asks whether the system can accommodate both — not by detecting the user's age, but by designing interactions that work across a range of expertise and comfort.

The test takes on special importance as ML systems move from research tools to everyday infrastructure. A medical imaging assistant used by radiologists, a legal document classifier used by clerks, an educational feedback system used by teachers — each involves humans-in-the-loop who did not sign up to be AI trainers. Designing for them is not a concession; it is the point.

:::{admonition} Design Principle :class: tip The Grandmother Test is a design constraint, not a target demographic. Systems that pass it are more robust to user diversity, more forgiving of expertise gaps, and more honest about what they expect from humans-in-the-loop. If a system

requires explanation before use, it is asking the human to do extra work. That work should be justified by proportionate benefit. :::

IML and Foundation Models

Modern IML increasingly leverages **pre-trained foundation models** {cite}`bommasani2021opportunities` as the base. Instead of training from scratch, users fine-tune a large pre-trained model with a small number of interactive examples. This can dramatically reduce the number of examples needed to reach useful performance — in favourable cases, as few as 5–50 examples rather than thousands, depending on how well the pre-trained representations match the target task {cite}`bommasani2021opportunities`.

Techniques enabling this include: - **Few-shot prompting:** Providing examples in the LLM's context window - **Adapter fine-tuning:** Updating small adapter modules while freezing the base model - **Parameter-efficient fine-tuning (PEFT):** LoRA, prefix tuning, and similar methods that allow fast, low-resource updates

Foundation models change the IML dynamic: humans are no longer teaching a blank-slate model from scratch but *steering* a model that already knows a great deal. The challenge shifts from "how to provide enough examples" to "how to specify exactly what we want differently from what the model already does."

> **See also:** The survey by {cite}`amershi2014power` remains the best overview of IML principles. For mixed-initiative systems specifically, see {cite}`allen1999mixed`. For anchoring effects in annotation, see {cite}`geva2019annotator` and {cite}`lingren2014evaluating`.

Reinforcement Learning from Human Feedback

No technique has done more to bring HITL ML into the mainstream than Reinforcement Learning from Human Feedback (RLHF). It is the mechanism behind InstructGPT {cite}`ouyang2022training` and a core component of instruction-following pipelines in many modern large language models {cite}`stiennon2020learning`. Understanding RLHF — not just as a recipe to follow, but as a principled approach to alignment — is essential for anyone working in modern AI.

The Alignment Problem

Large language models (LLMs) trained purely on next-token prediction optimize for a proxy objective: predict what text comes next in a corpus of human-written text. This objective is related to, but distinct from, what we actually want: responses that are helpful, accurate, safe, and aligned with human values.

The mismatch between the training objective and the desired behavior is called the **alignment problem** {cite}`russell2019human`. Concretely, a language model trained on internet text learns to: - Produce plausible-sounding continuations (which may be factually wrong) - Reflect the biases and harms present in training data - Be evasive or manipulative when this is what follows the prompt statistically

RLHF addresses alignment by making human preferences *part of the optimization objective*.

The RLHF Pipeline

RLHF proceeds in three stages:

```
Stage 1: Supervised Fine-Tuning (SFT)
  --> Collect demonstration data (human writes ideal responses)
  --> Fine-tune base LLM on demonstrations

Stage 2: Reward Model Training
  --> Collect pairwise preferences (human rates A vs B)
  --> Train reward model R(x, y) to predict human preferences

Stage 3: RL Fine-Tuning
  --> Fine-tune LLM using PPO/RL to maximize R(x, y)
  --> KL penalty prevents excessive deviation from SFT model
```

Stage 1: Supervised Fine-Tuning

Starting from a pre-trained base model π_0, we collect a dataset of (prompt, ideal response) pairs, written or selected by human contractors who follow detailed guidelines. The model is fine-tuned on these demonstrations using standard cross-entropy:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(x,y)\sim\mathcal{D}_{\text{demo}}}\left[\log \pi_\theta(y \mid x)\right]$$

The SFT model π_{SFT} is a much better starting point for RLHF than the raw pre-trained model.

Stage 2: Reward Model Training

For a set of prompts $\{x_i\}$, we generate K responses per prompt using π_{SFT} and present them to human labelers as pairwise comparisons: "Which response is better, A or B?"

The reward model r_ϕ is trained to predict these preferences. Under the **Bradley-Terry** model (Chapter 8), the probability that response y_w is preferred to y_l is:

$$P(y_w \succ y_l \mid x) = \sigma\left(r_\phi(x, y_w) - r_\phi(x, y_l)\right)$$

The reward model is trained to minimize the pairwise ranking loss:

$$\mathcal{L}_{\text{RM}}(\phi) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}_{\text{pref}}}\left[\log \sigma\left(r_\phi(x, y_w) - r_\phi(x, y_l)\right)\right]$$

The reward model is typically initialized from the SFT model with a scalar head replacing the final layer.

Stage 3: RL Fine-Tuning with PPO

With a trained reward model, we can use reinforcement learning to fine-tune the LLM. Each prompt x is a state; each response y is a trajectory of token choices; and the reward is $r_\phi(x, y)$.

The optimization objective includes a **KL divergence penalty** to prevent the model from drifting too far from the SFT baseline (which would lead to reward hacking {cite}`krakovna2020specification,gao2023`

$$\max_\theta \mathbb{E}_{x\sim\mathcal{D}, y\sim\pi_\theta(\cdot|x)}\left[r_\phi(x, y) - \beta \cdot \text{KL}\left[\pi_\theta(\cdot \mid x) \| \pi_{\text{SFT}}(\cdot \mid x)\right]\right]$$

The β parameter controls the KL penalty strength. Small β allows more optimization but risks reward hacking; large β keeps the model close to SFT but limits alignment gains.

Proximal Policy Optimization (PPO) {cite}`schulman2017proximal` is the standard algorithm for this stage, chosen for its stability relative to raw policy gradient methods.

A Simplified RLHF Demonstration

The full RLHF pipeline requires large-scale infrastructure. The following example demonstrates the key ideas at small scale.

```python
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from torch.nn import functional as F

torch.manual_seed(42)
rng = np.random.default_rng(42)

# ----------------------------------------------
# Toy setup: responses are 4-dimensional vectors
# "Quality" is known analytically (sum of positive values)
# We simulate a reward model learning this from pairwise feedback
# ----------------------------------------------

class RewardModel(nn.Module):
    def __init__(self, d=4, hidden=32):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(d, hidden), nn.ReLU(),
            nn.Linear(hidden, hidden), nn.ReLU(),
            nn.Linear(hidden, 1)
        )
    def forward(self, x):
        return self.net(x).squeeze(-1)

def true_quality(x):
    """The hidden ground-truth reward function."""
    return x.sum(dim=-1) + 0.5 * (x ** 2).mean(dim=-1)

# Generate pairwise preference data
N_PAIRS = 500
X1 = torch.randn(N_PAIRS, 4)
X2 = torch.randn(N_PAIRS, 4)
q1, q2 = true_quality(X1), true_quality(X2)
# Human prefers X1 when q1 > q2 (with some noise)
noise = torch.randn(N_PAIRS) * 0.5
preferred_1 = ((q1 - q2 + noise) > 0).float()

# Train reward model
rm = RewardModel(d=4, hidden=32)
optimizer = optim.Adam(rm.parameters(), lr=3e-3)

losses = []
for epoch in range(200):
    r1 = rm(X1)
    r2 = rm(X2)
    # Bradley-Terry loss
    logit = r1 - r2
    loss = F.binary_cross_entropy_with_logits(logit, preferred_1)
    optimizer.zero_grad(); loss.backward(); optimizer.step()
```

```
    losses.append(loss.item())

# Evaluate: does the reward model agree with true quality?
X_eval = torch.randn(1000, 4)
with torch.no_grad():
    r_pred = rm(X_eval).numpy()
    r_true = true_quality(X_eval).numpy()

corr = np.corrcoef(r_pred, r_true)[0, 1]
print(f"Reward model correlation with true quality: {corr:.4f}")
print(f"Final training loss: {losses[-1]:.4f}")

import matplotlib.pyplot as plt
plt.figure(figsize=(8, 3))
plt.subplot(1, 2, 1)
plt.plot(losses, color='#2b3a8f', linewidth=1.5)
plt.xlabel("Epoch"); plt.ylabel("Pairwise loss")
plt.title("Reward Model Training")

plt.subplot(1, 2, 2)
plt.scatter(r_true[:200], r_pred[:200], alpha=0.4, s=15, color='#2b3a8f')
plt.xlabel("True quality"); plt.ylabel("Predicted reward")
plt.title(f"Reward Model vs. Truth (r={corr:.3f})")
plt.tight_layout()
plt.savefig('reward_model.png', dpi=150)
plt.show()
```

Challenges in RLHF

Reward Hacking

A key failure mode: the policy finds ways to get high reward from the reward model that do not correspond to genuinely good behavior. For example, an LLM might learn to produce responses that are flattering or confidently worded (which labelers tend to rate highly) rather than accurate.

Reward hacking is more likely when: - The reward model is trained on insufficient preference data - The policy is allowed to diverge far from the SFT baseline (small β) - The reward model distribution shifts during PPO training

Mitigation strategies: KL penalty, iterated reward model training, diverse evaluation, constitutional AI constraints.

Evaluator Bias

Human labelers have systematic biases. They tend to prefer longer responses (verbosity bias), more confident-sounding text (confidence bias), and responses that agree with their prior beliefs. These biases propagate into the reward model.

The famous sycophancy failure of RLHF models — where the model tells users what they want to hear rather than what is true — is partly a result of evaluator preference for agreeable responses.

Scalable Oversight

For complex tasks, humans cannot reliably judge which AI response is correct. A labeler comparing two long mathematical proofs or two code implementations may simply pick the more readable one, regardless of correctness. **Scalable oversight** is the open research problem of designing evaluation procedures that remain reliable as task complexity grows {cite}`bowman2022measuring`.

Constitutional AI (RLAIF)

Constitutional AI {cite}`bai2022constitutional`, developed at Anthropic, reduces reliance on human labelers by using the AI itself to generate preference labels guided by a set of principles (a "constitution"). The process:

1. Generate responses to potentially harmful prompts
2. Use an AI critic to evaluate responses against constitutional principles
3. Revise responses guided by AI feedback (RLAIF — RL from AI Feedback)
4. Train a reward model on AI-generated preferences
5. Fine-tune with RLHF using this reward model

RLAIF can generate preference data at a far larger scale than human labeling, and allows fine-grained control over the values encoded in the reward model.

> **See also:** The original InstructGPT paper {cite}`ouyang2022training` describes the first large-scale application of RLHF to LLMs. The foundational RLHF for deep RL work is {cite}`christiano2017deep`. PPO is described in {cite}`schulman2017proximal`. Constitutional AI is from {cite}`bai2022constitutional`.

Learning from Demonstrations

When a task is difficult to specify but easy to demonstrate, it can be more efficient to teach by example than to define by rule. A human expert shows a robotic arm how to grasp an object; a programmer's interaction with their IDE provides a sequence of correct edits; a chess grandmaster plays a game. **Learning from demonstrations** extracts a policy from such behavioral data, avoiding the need for hand-crafted reward functions or explicit task specifications.

Behavioral Cloning

The simplest approach is **behavioral cloning (BC)**: treat the demonstration as supervised data and learn a mapping from states to actions.

Given a dataset of state-action pairs $\mathcal{D} = \{(s_i, a_i)\}$ from an expert demonstrator, we fit a policy $\pi_\theta(a \mid s)$ by minimizing the negative log-likelihood:

$$\mathcal{L}_{\text{BC}}(\theta) = -\frac{1}{|\mathcal{D}|} \sum_{(s,a)\in\mathcal{D}} \log \pi_\theta(a \mid s)$$

This is precisely standard supervised learning applied to sequential data.

The Covariate Shift Problem

BC has a fundamental weakness: **distribution shift** between training and deployment. The expert's demonstrations cover states visited by the expert. But during deployment, the learned policy may make slightly different decisions, leading it to states the expert never visited — states where the policy has no supervision and may fail badly.

Crucially, errors **compound**: a small deviation leads to an unfamiliar state, where a slightly wrong action leads to an even more unfamiliar state, and so on. Performance degrades as $O(T^2\epsilon)$ where T is the episode length and ϵ is the error rate at each step — much worse than the $O(T\epsilon)$ degradation of an oracle policy {cite}`ross2010efficient`.

> **Example: Autonomous Driving** :class: note
>
> A behavioral cloning model for lane-keeping trained on human driving data performs well on straight roads (states near the training distribution). But the moment it drifts slightly — a state no human driver would be in because they would have already corrected — it has no data to guide it and may drive off the road.

```
Algorithm DAgger:
  Initialize: D <- {} (empty dataset)
  Train initial policy pi_1 on M expert demonstrations

  for iteration i = 1, 2, ..., N:
    1. Run pi_i in the environment to collect states {s_1, ..., s_t}
    2. Query expert for actions on each visited state: a_t = pi*(s_t)
    3. Aggregate: D <- D u {(s_1, a_1), ..., (s_t, a_t)}
    4. Train pi_{i+1} by supervised learning on D
```

DAgger achieves $O(T\epsilon)$ regret — the same as an oracle policy — because the training distribution converges to match the deployment distribution.

The key requirement is that the expert can be queried on any state, including states the expert would never naturally visit. This is feasible in simulation (ask the expert to correct the robot from an unusual configuration) but can be challenging or unsafe in real physical systems.

Inverse Reinforcement Learning

Sometimes the expert's behavior is better understood not as a sequence of actions to mimic but as the outcome of optimizing an unknown reward function. **Inverse Reinforcement Learning (IRL)** {cite}`ng2000algorithms` recovers this latent reward function from demonstrations.

Given demonstrations $\tau = \{(s_1, a_1), \dots, (s_T, a_T)\}$, IRL finds a reward function $R(s, a)$ such that the expert's policy is optimal under R.

The appeal of IRL over BC: if we recover the true reward function, we can re-optimize it in new environments, with different dynamics, or with improved planners — generalizing far beyond the demonstrated scenarios.

Maximum Entropy IRL

MaxEnt IRL {cite}`ziebart2008maximum` resolves the IRL ambiguity problem (there are many reward functions consistent with any set of demonstrations) by choosing the reward function that, while consistent with the demonstrated behavior, leads to a distribution over trajectories with *maximum entropy*. Trajectories are distributed as:

$$P(\tau \mid R) \propto \exp\left(\sum_t R(s_t, a_t)\right)$$

The learning objective matches the expert's observed feature expectations $\mu_E = \mathbb{E}_{\tau\sim\pi^*}[\phi(\tau)]$ to the model's feature expectations $\mu_\theta = \mathbb{E}_{\tau\sim\pi_\theta}[\phi(\tau)]$.

GAIL: Generative Adversarial Imitation Learning

GAIL {cite}`ho2016generative` bypasses reward function learning entirely, using a GAN-like formulation to directly match the state-action distribution of the expert.

A discriminator D_ψ is trained to distinguish expert state-action pairs $(s, a) \sim \pi^*$ from policy state-action pairs $(s, a) \sim \pi_\theta$:

$$\mathcal{L}_D = -\mathbb{E}_{\pi^*}[\log D_\psi(s, a)] - \mathbb{E}_{\pi_\theta}[\log(1 - D_\psi(s, a))]$$

The generator (the policy π_θ) is trained to fool the discriminator — i.e., to produce state-action pairs that look like the expert's. The discriminator's output $\log D_\psi(s, a)$ serves as a reward signal for the policy.

GAIL achieves expert-level performance on continuous control benchmarks with far fewer demonstrations than BC, and generalizes better than MaxEnt IRL in complex environments.

Behavioral Cloning in NLP: A Practical Example

```python
import numpy as np
import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset

torch.manual_seed(42)

# ----------------------------------------------
# Toy NLP task: rewriting sentences to be more formal
# We simulate this as a simple sequence transformation
# In practice: fine-tuning a seq2seq model on expert rewrites
# ----------------------------------------------

class SimpleSeq2Seq(nn.Module):
    def __init__(self, vocab_size=100, embed_dim=32, hidden_dim=64):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
        self.encoder = nn.GRU(embed_dim, hidden_dim, batch_first=True)
        self.decoder = nn.GRU(embed_dim, hidden_dim, batch_first=True)
        self.proj = nn.Linear(hidden_dim, vocab_size)
        self.hidden_dim = hidden_dim

    def forward(self, src, tgt):
        src_emb = self.embed(src)
        _, hidden = self.encoder(src_emb)
        tgt_emb = self.embed(tgt)
        out, _ = self.decoder(tgt_emb, hidden)
        return self.proj(out)

# Generate synthetic demonstration data
VOCAB = 100
rng = np.random.default_rng(42)
N, SEQ_LEN = 1000, 12

src_seqs = torch.tensor(rng.integers(1, VOCAB, (N, SEQ_LEN)), dtype=torch.long)
# "Expert" transformation: shift tokens by 1 (toy formalization)
```

```
tgt_seqs = torch.clamp(src_seqs + 1, 1, VOCAB - 1)
tgt_in  = torch.cat([torch.ones(N, 1, dtype=torch.long), tgt_seqs[:, :-1]], dim=1)

dataset = TensorDataset(src_seqs, tgt_in, tgt_seqs)
loader = DataLoader(dataset, batch_size=64, shuffle=True)

model = SimpleSeq2Seq(vocab_size=VOCAB)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
criterion = nn.CrossEntropyLoss(ignore_index=0)

# Behavioral cloning training
train_losses = []
for epoch in range(20):
    epoch_loss = 0
    for src, tgt_i, tgt_o in loader:
        logits = model(src, tgt_i)
        loss = criterion(logits.reshape(-1, VOCAB), tgt_o.reshape(-1))
        optimizer.zero_grad(); loss.backward(); optimizer.step()
        epoch_loss += loss.item()
    train_losses.append(epoch_loss / len(loader))

print(f"Initial loss: {train_losses[0]:.3f}")
print(f"Final loss:   {train_losses[-1]:.3f}")

# Evaluate: check token accuracy on held-out examples
model.eval()
with torch.no_grad():
    src_test = src_seqs[-100:]
    tgt_test_in = tgt_in[-100:]
    tgt_test_out = tgt_seqs[-100:]
    logits = model(src_test, tgt_test_in)
    preds = logits.argmax(dim=-1)
    acc = (preds == tgt_test_out).float().mean().item()
    print(f"Token accuracy on held-out set: {acc:.3f}")
```

Comparison of Imitation Learning Methods

Method	Requires reward?	Expert queried online?	Generalizes to new dynamics?	Complexity
Behavioral Cloning	No	No	Poorly (distribution shift)	Low
DAgger	No	Yes	Moderately	Medium
MaxEnt IRL	Recovers it	No	Well	High
GAIL	No	No	Well	High

Applications

Robotics. Teaching robots to manipulate objects, navigate environments, or perform household tasks. Physical demonstrations are collected via teleoperation or kinesthetic teaching.

Autonomous driving. Early self-driving systems such as ALVINN {cite}`pomerleau1989alvinn` and NVIDIA's DAVE relied heavily on behavioral cloning from human driving data.

Game AI. Imitation learning on human gameplay bootstraps agents before RL fine-tuning. AlphaStar trained on human replays before RL; this approach is common when human-level demonstrations are available.

Code generation. Language model fine-tuning on high-quality code demonstrations (GitHub Copilot, Codex) is a form of behavioral cloning.

Clinical decision support. Learning from expert physician decision sequences for complex protocols.

> **See also:** The foundational BC/DAgger analysis is in {cite}`ross2011reduction`. MaxEnt IRL is from {cite}`ziebart2008maximum`. GAIL is from {cite}`ho2016generative`. For a comprehensive survey of imitation learning, see {cite}`osa2018algorithmic`.

Learning from Comparisons and Rankings

Asking humans to assign an absolute quality score to an output is hard. What is the numerical quality of this essay, on a scale from 1 to 10? The question is ill-posed: humans lack a stable internal scale, and their scores are heavily influenced by anchoring, context, and fatigue.

Asking humans to *compare* two outputs is much easier: which essay is better, A or B? Comparative judgments are more consistent, more reliable, and tap into human preferences more directly than absolute ratings. This chapter covers the mathematical foundations and practical applications of learning from comparisons and rankings.

Why Comparisons Are Better Than Ratings

Psychological Grounding

The superiority of comparative judgments has a long history in psychometrics. Thurstone's Law of Comparative Judgment {cite}`thurstone1927law` (1927) showed that even when humans have inconsistent absolute judgments, their relative judgments follow a consistent probabilistic law. Comparisons exploit the fact that humans are far better at *relative* perception than absolute calibration.

Statistical Efficiency

Each pairwise comparison provides information about the *relative* positions of two items on the quality scale. With K items, $K-1$ comparisons can rank all items; only $O(\log K)$ comparisons are needed to find the top item. Absolute ratings typically require more judgments to achieve the same precision.

Scalability

For generative models, the number of distinct outputs is effectively infinite. Rating an output absolutely requires establishing a shared scale across all outputs; comparing outputs requires only local, relative judgments that are naturally calibrated to each other.

The Bradley-Terry Model

The dominant probabilistic model for pairwise comparisons is the **Bradley-Terry (BT) model** {cite}`bradley1952rank`. Each item i has a latent quality score $\alpha_i \in \mathbb{R}$. The probability that item

i is preferred to item j in a direct comparison is:

$$P(i \succ j) = \frac{e^{\alpha_i}}{e^{\alpha_i} + e^{\alpha_j}} = \sigma(\alpha_i - \alpha_j)$$

where σ is the logistic sigmoid. This is equivalent to assuming that the perceived quality of item i is $\alpha_i + \epsilon$ where ϵ is a standard logistic noise term.

Identifiability

The BT model is identifiable up to a translation: if α is a solution, so is $\alpha + c$ for any constant c. A standard convention is to fix one score (e.g., $\alpha_0 = 0$) or to constrain $\sum_i \alpha_i = 0$. The scores are only identified when the **comparison graph** (nodes = items, edges = observed pairs) is **connected** — if the graph has disconnected components, the relative scores across components are undefined.

Parameter Estimation

Given a dataset of pairwise comparisons $\mathcal{D} = \{(i, j, y_{ij})\}$ where $y_{ij} = 1$ if i was preferred to j, the log-likelihood is:

$$\mathcal{L}(\alpha) = \sum_{(i,j,y_{ij}) \in \mathcal{D}} \left[y_{ij} \log \sigma(\alpha_i - \alpha_j) + (1 - y_{ij}) \log \sigma(\alpha_j - \alpha_i) \right]$$

This is a concave function of α and can be maximized via gradient ascent or Newton's method.

```python
import numpy as np
from scipy.optimize import minimize
from scipy.special import expit  # sigmoid

rng = np.random.default_rng(42)

# ----------------------------------------------
# Simulate Bradley-Terry: 8 items with true quality scores
# Generate pairwise comparisons and recover the scores
# ----------------------------------------------

N_ITEMS = 8
true_alpha = rng.normal(0, 1, N_ITEMS)
print(f"True quality ranking: {np.argsort(true_alpha)[::-1]}")

# Generate comparisons: every pair compared 5 times
comparisons = []
for i in range(N_ITEMS):
    for j in range(i + 1, N_ITEMS):
        for _ in range(5):
            p_i_wins = expit(true_alpha[i] - true_alpha[j])
            winner = i if rng.random() < p_i_wins else j
            loser  = j if winner == i else i
            comparisons.append((winner, loser))

print(f"Total comparisons: {len(comparisons)}")
```

```python
def neg_log_likelihood(alpha, comparisons):
    """Bradley-Terry negative log-likelihood."""
    alpha = np.array(alpha)
    loss = 0.0
    for winner, loser in comparisons:
        log_prob = np.log(expit(alpha[winner] - alpha[loser]) + 1e-10)
        loss -= log_prob
    return loss

def neg_grad(alpha, comparisons):
    alpha = np.array(alpha)
    grad = np.zeros(len(alpha))
    for winner, loser in comparisons:
        p = expit(alpha[winner] - alpha[loser])
        grad[winner] -= (1 - p)
        grad[loser]  -= (-p)
    return grad

# Fix alpha[0] = 0 to resolve scale ambiguity
result = minimize(
    lambda a: neg_log_likelihood(np.concatenate([[0.0], a]), comparisons),
    x0=np.zeros(N_ITEMS - 1),
    jac=lambda a: neg_grad(np.concatenate([[0.0], a]), comparisons)[1:],
    method='L-BFGS-B'
)
est_alpha = np.concatenate([[0.0], result.x])

# Compare true vs estimated ranking
true_rank = np.argsort(true_alpha)[::-1]
est_rank  = np.argsort(est_alpha)[::-1]

print(f"\nTrue ranking (item indices): {true_rank}")
print(f"Estimated ranking:           {est_rank}")
rank_corr = np.corrcoef(true_alpha, est_alpha)[0, 1]
print(f"Correlation with true scores: {rank_corr:.4f}")
```

Thurstone's Model

Thurstone's model {cite}`thurstone1927law` is closely related to Bradley-Terry but uses a Gaussian rather than logistic noise:

$$P(i \succ j) = \Phi\left(\frac{\alpha_i - \alpha_j}{\sqrt{2}\sigma}\right)$$

where Φ is the standard normal CDF. When $\sigma = 1/\sqrt{2}$, this becomes equivalent to BT with a minor scaling difference. In practice, the two models give nearly identical results.

Rank Aggregation

When each annotator provides a complete ranking of K items (rather than pairwise comparisons), the problem is **rank aggregation**: combine multiple ranked lists into a consensus ranking.

Borda count: Each item receives a score equal to the number of items ranked below it in each annotator's ranking. Scores are summed across annotators. Simple and robust.

Kemeny–Young: Find the ranking that minimizes the sum of pairwise disagreements (the Kendall tau distance) with each annotator's ranking. This is NP-hard for large K but tractable for small sets.

RankNet / ListNet: Neural approaches that learn a scoring function from ranked lists, enabling generalization to unseen items.

Dueling Bandits

In **online** preference learning, items arrive in a stream and we must decide which pairs to compare, balancing exploration (learning about unknown items) and exploitation (presenting high-quality items). This is the **dueling bandit** problem {cite}`yue2009interactively`.

Key algorithms: - **Doubler:** Maintains a champion item; challenges it with random competitors - **RUCB (Relative Upper Confidence Bound):** Computes UCB-style confidence intervals for each item's probability of beating each other item - **MergeRank:** Combines tournament-style comparison with UCB

Dueling bandits are used in online recommendation systems (which article to show next, given implicit feedback) and interactive preference elicitation for RLHF data collection.

Preference Learning for Language Models

In the context of RLHF (Chapter 6), the Bradley-Terry model is used to train the reward model. An important variant is **Direct Preference Optimization (DPO)** {cite}`rafailov2023direct`, which shows that the RLHF objective can be optimized directly from preference data without training a separate reward model:

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x,y_w,y_l)\sim\mathcal{D}}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta\log\frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right]$$

DPO is simpler than full RLHF (no PPO training loop, no reward model) while achieving comparable or better results on many benchmarks {cite}`rafailov2023direct`. It has become a widely adopted alternative to PPO-based RLHF for instruction-following fine-tuning, though both approaches remain in active use and their relative strengths are task-dependent.

Collecting High-Quality Preference Data

The quality of preference data determines the quality of the reward model. Key considerations:

Prompt diversity. Preference data should cover the full distribution of prompts the model will encounter in deployment. Gaps in coverage lead to unreliable reward model behavior in those regions.

Response diversity. Comparing two very similar responses provides little information. The compared responses should differ enough for annotators to have a clear preference.

Annotator agreement. Low inter-annotator agreement suggests the comparison criteria are ambiguous. Measure agreement (Cohen's κ) and revise guidelines when it is below acceptable thresholds.

Calibration. Annotators should understand *why* one response is better: helpfulness, accuracy, safety, style? Tasks that bundle multiple criteria tend to produce inconsistent preferences. It is often better to collect preferences on each criterion separately.

> **See also:** Bradley-Terry model: {cite}`bradley1952rank`. Thurstone: {cite}`thurstone1927law`. Dueling bandits: {cite}`yue2009interactively`. Direct Preference Optimization (DPO): {cite}`rafailov2023direct`.

HITL in Natural Language Processing

Natural language is the domain where HITL ML has arguably had the greatest impact — and where its deepest conceptual difficulties surface. Language is inherently social: its meaning is constructed by human communities, its pragmatics depend on context and intent, and its quality can only be judged by human readers. But this also means that NLP annotation is not merely a process of collecting observations. It is a process of *constructing* categories.

The Constitutive Problem in NLP Annotation

In medical imaging, there is a ground truth: a tumor is present or it is not. The radiologist's label may be uncertain, but it is attempting to track something real. NLP annotation is often fundamentally different. When an annotator marks a tweet as "toxic," there is no toxic-molecule in the tweet that we are attempting to detect. **The label constitutes the category.**

This has profound consequences that are frequently underappreciated:

The annotation workforce defines the phenomenon. A label schema for "offensive speech" encodes the sensibilities of whoever designed it and whoever applied it. A team of English-speaking annotators from a single demographic background, working under guidelines written by a corporate policy team, produces a dataset that reflects that team's understanding of offense — not some universal human standard. Models trained on such data will exhibit those same implicit boundaries.

Guidelines are theory, whether or not they acknowledge it. Every annotation schema makes claims about ontology. Deciding that "irony" and "sarcasm" are the same class is a theoretical claim, not a neutral convenience. Deciding to label "anger" as a single class rather than distinguishing "righteous anger" from "hostile anger" collapses a distinction that may matter for the downstream task. These decisions are made under production pressure and rarely revisited.

Label instability over time. Social language evolves. A toxicity model trained in 2018 will misclassify content from 2024 not because it is statistically undertrained, but because the social meaning of certain terms has changed. NLP annotation is not sampling from a static distribution — it is sampling from a moving target that the act of labeling partially helps to constitute.

:::{admonition} The Annotation Artifact Problem :class: important

Geva et al. {cite}`geva2019annotator` demonstrated that NLI (Natural Language Inference) datasets contain systematic artifacts introduced by the annotation process itself. Annotators asked to write "entailment" hypotheses for a given premise tend to use certain syntactic patterns; annotators writing "contradiction" hypotheses use others. Models learn to classify

based on these artifacts rather than the intended semantic relationship — they solve the annotation task, not the underlying NLP task.

This is not carelessness. It is an inherent consequence of having humans construct examples to fit a label. The HITL process inserts a systematic signal that was never intended to be in the data. :::

Text Classification Annotation

The simplest NLP annotation task is assigning a category to a text document. Sentiment analysis, topic classification, intent detection, and spam filtering are all classification tasks.

Challenges specific to text classification:

Subjectivity. Categories like "toxic" or "offensive" are inherently subjective and vary across cultural contexts, annotator backgrounds, and platform context. Across annotators, perceptions of offensiveness differ significantly by demographic — a fact with direct implications for fairness {cite}`blodgett2020language`.

Label ambiguity. Many documents belong to multiple categories or fall on a boundary. A review that is 60% positive and 40% negative is genuinely ambiguous, not mislabeled. Forcing a single label discards real information (see Chapter 13 on soft labels and annotator disagreement).

Label granularity. A 2-class sentiment model may be sufficient for social media monitoring; a 7-point scale may be needed for product feedback analysis. The right granularity depends on the downstream task, but is usually fixed before the annotation — a consequential design decision made with insufficient data.

Named Entity Recognition

NER annotation requires identifying spans of text and assigning an entity type (PERSON, ORGANIZATION, LOCATION, etc.). The annotation is more complex than document classification for several reasons:

Span boundaries are ambiguous. In "Apple CEO Tim Cook announced…", does the PERSON entity span "Tim Cook" or "Apple CEO Tim Cook"? Guidelines must explicitly address these cases, and inter-annotator agreement on spans is consistently lower than on types.

Type assignment requires world knowledge. "Apple" is ORG in one context, PRODUCT in another, and arguably neither in "apple pie." Annotators need sufficient domain knowledge to make correct type assignments.

Nested entities. "The University of California, Berkeley" contains ORGANIZATION nested within LOCATION. Standard BIO tagging cannot represent nested entities; more complex schemes (e.g., BIOES, or graph-based formats) are required.

Annotation efficiency. Pre-annotation with a baseline NER model significantly speeds up annotation by allowing annotators to correct predictions rather than annotate from scratch. In one study of clinical NER, throughput increases of 30–60% were observed {cite}`lingren2014evaluating`; the magnitude of such gains depends heavily on baseline model quality and domain.

Relation Extraction and Semantic Annotation

Beyond identifying entities, relation extraction requires annotating *relationships* between entities:

- Annotators must understand both entities and the predicate connecting them
- Relation types have complex semantic distinctions (WORKS_AT vs. EMPLOYED_BY vs. FOUNDED)
- Negative examples (entity pairs with no relation) are far more common than positive ones

Inter-annotator agreement for relation extraction tends to be lower than for classification tasks. For well-defined schemas, κ values in the range 0.65–0.80 are commonly reported {cite}`pustejovsky2012natural`; for complex semantic relations (event causality, temporal relations), agreement can fall considerably lower, depending on schema design and annotator training.

Machine Translation Post-Editing (MTPE)

Machine translation post-editing is a mature form of HITL NLP. A human translator corrects the output of an MT system rather than translating from scratch:

Light post-editing (LPE): Only critical errors are corrected. Suitable when translation quality requirements are moderate.

Full post-editing (FPE): The output is corrected to publication quality. The edited output typically becomes training data for further MT improvement — a genuine human-in-the-loop refinement cycle.

HTER (Human-targeted Translation Edit Rate): A metric that measures the edit distance between the MT output and the post-edited translation, normalized by sentence length {cite}`graham2015accurate`. As a rough practitioner rule of thumb, HTER below roughly 0.3 is often considered good MT output; above 0.5, translation from scratch may be faster — though these thresholds vary by domain and language pair.

Conversational AI and Dialogue Annotation

Annotating dialogue introduces temporal structure:

- **Turn-level annotation:** label each turn (intent, sentiment, quality)
- **Dialogue-level annotation:** assess overall coherence, task success, user satisfaction
- **Interaction trace annotation:** identify specific failure moments in a conversation

HITL is especially important in dialogue because model failures are often subtle and cumulative: a factual error in turn 3 may not become apparent until turn 7. Human annotators tracing conversations can identify these long-range failure patterns that automated metrics entirely miss.

Programmatic Labeling and Weak Supervision

When labeled data is scarce, **weak supervision** allows using multiple noisy, overlapping labeling functions to generate probabilistic labels at scale. **Snorkel** {cite}`ratner2017snorkel` is the canonical framework:

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score

rng = np.random.default_rng(42)

# ---------------------------------------------------------
# Toy weak supervision: sentiment classification
# 5 labeling functions (LFs) with different coverage/accuracy
# Label: +1 (positive), -1 (negative), 0 (abstain)
# ---------------------------------------------------------

N = 1000
true_labels = rng.choice([-1, 1], size=N)
X_features = np.column_stack([
    true_labels * 0.8 + rng.normal(0, 0.5, N),
    rng.normal(0, 1, N),
    rng.normal(0, 1, N),
])

def make_lf(accuracy, coverage, seed):
    rng_lf = np.random.default_rng(seed)
    votes = np.zeros(N, dtype=int)
    active = rng_lf.random(N) < coverage
    correct = rng_lf.random(N) < accuracy
    votes[active & correct]  = true_labels[active & correct]
    votes[active & ~correct] = -true_labels[active & ~correct]
    return votes

LFs = np.column_stack([
    make_lf(accuracy=0.85, coverage=0.60, seed=1),
    make_lf(accuracy=0.78, coverage=0.45, seed=2),
    make_lf(accuracy=0.70, coverage=0.80, seed=3),
    make_lf(accuracy=0.90, coverage=0.30, seed=4),
    make_lf(accuracy=0.60, coverage=0.90, seed=5),
])

def majority_vote(LF_matrix):
    labels = []
    for i in range(LF_matrix.shape[0]):
        votes = LF_matrix[i][LF_matrix[i] != 0]
        labels.append(0 if len(votes) == 0 else int(np.sign(votes.sum())))
    return np.array(labels)

mv_labels = majority_vote(LFs)
covered = mv_labels != 0
```

```
print(f"Coverage:                    {covered.mean():.1%}")
print(f"MV accuracy (covered):       {(mv_labels[covered] == true_labels[covered]).mean():.3f

X_train, y_train = X_features[covered], mv_labels[covered]
X_test,  y_test  = X_features[~covered], true_labels[~covered]

if len(X_train) > 10 and len(X_test) > 10:
    clf = LogisticRegression().fit(X_train, y_train)
    preds = clf.predict(X_test)
    print(f"F1 on uncovered test set:    {f1_score(y_test, preds):.3f}")
```

RLHF for Language Models: The Annotation Reality

Chapter 6 covered RLHF technically. From an NLP perspective, the annotation task is harder than it appears from the outside.

What annotators are actually asked to do — compare two model outputs and indicate which is "better" — sounds simple. In practice, "better" is an underspecified construct that annotators resolve using personal heuristics. Some weight fluency heavily; others weight factual accuracy; others penalize verbosity. Without tight guidelines, the resulting preference dataset reflects not human values in the abstract but the particular resolution strategies of the annotation workforce employed.

The key annotation dimensions are:

- *Helpfulness:* Does the response answer the question well? Is it informative, clear, and appropriately detailed?
- *Factuality:* Is the response factually accurate? This requires evaluators to have domain knowledge — a requirement that creates serious quality problems at scale, since generalist annotators cannot verify specialist claims.
- *Harmlessness:* Does the response avoid toxic, biased, harmful, or inappropriate content? These judgments require detailed guidelines because "harmful" is highly context-dependent and shifts across cultures, time, and platform context.
- *Calibration:* Does the response express appropriate uncertainty? Annotators systematically prefer responses that sound confident, which creates a reward signal against appropriate epistemic humility.

The interaction between criteria is complex: a maximally helpful response to a dangerous question may be maximally harmful. Guidelines must specify how to trade off competing criteria — and those trade-offs are effectively policy decisions, not annotation decisions. The annotation workforce is making policy.

Scale concentrates demographic influence. RLHF for large models involves relatively small annotation workforces (hundreds to low thousands) making billions of downstream decisions. The demographic and cultural biases of that workforce propagate into the model's behavior at scale in a way that would not happen if annotation were more distributed. This is one of the least-discussed structural problems in the current RLHF pipeline.

The Annotation–Model Feedback Loop

In NLP more than in any other domain, the annotation and model development processes become entangled over time:

1. Annotators are calibrated using a model's existing outputs as reference (often implicitly).
2. The reward model learns what annotations tend to prefer.
3. The generator is fine-tuned to produce outputs that get high reward.
4. Those outputs influence what "good" looks like in subsequent annotation rounds.

This feedback loop is not inherently pathological — it is what allows RLHF to converge — but it means that the model's behavior is shaped by a moving target that the annotation process itself helps to move. Distinguishing what the model learned because it reflects human preferences from what the model learned because it learned to pattern-match annotator heuristics is empirically difficult.

There is no clean solution. The best current practice is to monitor for drift using held-out preference judgments collected at regular intervals, and to treat annotation guideline version as an experimental variable.

Evaluation of Generative NLP Models

Unlike classification models with a clear accuracy metric, evaluating generation quality requires human judgment:

Evaluation method	Description	Cost
Direct assessment (DA)	Annotators rate quality on a scale	Medium
Comparative judgment	Annotators compare two outputs	Low
MQM (Multidimensional Quality Metrics)	Structured error taxonomy	High
RLHF preference	Preference labels used for training	Medium
LLM-as-judge	LLM scores outputs (correlates moderately with human)	Very low
BERTScore, BLEU	Automatic metrics (imperfect correlation with human judgment)	Very low

Automatic metrics (BLEU for MT, ROUGE for summarization) are cheap but imperfectly correlated with human quality judgments {cite}`reiter2018structured`. LLM-as-judge approaches show moderate agreement with human annotators {cite}`zheng2023judging` and are increasingly used for rapid iteration, but should not replace human evaluation for final assessments. For decisions with real stakes, human assessment remains necessary.

> **See also:** Snorkel weak supervision: {cite}`ratner2017snorkel`. NLP annotation guidelines: {cite}`pustejovsky2012natural`. Annotation artifacts in NLI: {cite}`geva2019annotator`. Annotator bias and NLP datasets: {cite}`blodgett2020language`. For evaluation of generative models: {cite}`reiter2018structured`.

HITL in Computer Vision

Computer vision provides some of the most visually intuitive examples of HITL ML. The ImageNet challenge, built on 14 million human-labeled images, launched the deep learning era. Medical imaging annotation by radiologists powers diagnostic AI. Autonomous vehicles depend on millions of human-labeled frames to learn to perceive the world.

What is easy to miss: these are not simply cases of humans providing ground truth. They are cases of humans constructing datasets that embed specific perceptual, cultural, and operational choices — choices that become visible only later, when models fail in predictable ways.

How Annotation Choices Become Model Biases

The standard framing treats annotation as data collection: humans observe the world and record what they see. The more accurate framing is that annotation is *dataset design*: humans decide what categories to use, where to draw boundaries, which edge cases to include, and how to resolve ambiguity — and all of those decisions shape what the trained model will perceive.

The ImageNet Case

ImageNet {cite}`russakovsky2015imagenet` is the most consequential dataset in computer vision history. Its label set derives from WordNet synsets, chosen primarily for being numerous and semantically distinct. Several consequences of this design choice emerged years later:

- **Person categories encoded demographic associations.** Early versions of ImageNet's synset labels for people included many that would now be considered derogatory or biased, reflecting both the historical WordNet source and the annotation workforce's implicit decisions about which labels to apply to which images {cite}`yang2020towards`. Labels applied to images of people encoded race, gender, and class associations that propagated directly into model embeddings.
- **Fine-grained species taxonomy, coarse-grained of almost everything else.** ImageNet can distinguish 120 dog breeds but collapses enormous variation in tools, vehicles, food, and furniture into single categories. This was a consequence of following WordNet's structure, not a deliberate choice about what matters. Models trained on ImageNet exhibit the same asymmetric precision.
- **Western, English-speaking visual defaults.** The images were collected primarily from Flickr and Internet searches using English-language queries. The resulting distribution skews heavily toward the visual environment and cultural objects of wealthy, English-speaking countries.

None of these were errors. They were annotation design choices made quickly at scale, often by people who did not anticipate how the dataset would be used. The lesson is not that ImageNet should have been built differently (though it should have), but that **annotation design is model design**, and it should be treated with the same care.

:::{admonition} Annotation schema is a theory of the world :class: note

Every label taxonomy makes claims about what distinctions matter. Choosing to separate "car" from "truck" while collapsing all sedans into one class is a theoretical claim about which distinctions are semantically relevant. Choosing to annotate "person" as a single class regardless of pose, clothing, or activity is a different theoretical claim. Models trained on these schemas will make the same distinctions, and no more — they will not generalize beyond the categories they were trained to distinguish. :::

Image Classification Annotation

The simplest CV annotation task is assigning one or more labels to an entire image.

Label hierarchy. The label "dog" is a child of "animal" in a semantic hierarchy. Models trained on flat taxonomies may not generalize well across levels of abstraction. ImageNet uses a synset-based hierarchy that allows evaluation at multiple levels of specificity.

Multi-label ambiguity. A street scene may contain a car, a person, a bicycle, and a traffic light simultaneously. Deciding which labels to include requires clear guidelines about relevance thresholds.

Long-tail distributions. Natural image datasets follow a power law: a few categories are extremely common; most are rare. Active learning is particularly valuable for long-tail categories where random sampling yields only a handful of examples.

Object Detection: Bounding Box Annotation

Object detection requires annotators to draw axis-aligned bounding boxes around instances of each object class. This introduces geometric precision requirements and significant edge cases.

Annotation quality metrics:

IoU (Intersection over Union) measures the overlap between an annotated box and a reference box:

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

IoU ≥ 0.5 is the standard threshold for a "correct" detection in PASCAL VOC; COCO {cite}`lin2014microsoft` uses a range of thresholds from 0.5 to 0.95, which is considerably more demanding and more informative.

Annotation edge cases that must be resolved in guidelines: - Occluded objects: annotate visible portion or extrapolate full extent? - Truncated objects (partially outside the frame): include or exclude? - Crowd regions: use a special "crowd" annotation or annotate individual instances?

Each of these decisions changes what "correct detection" means — and therefore changes what the model is trained to do.

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as patches

def compute_iou(boxA, boxB):
    """Compute IoU between two boxes [x1, y1, x2, y2]."""
    xA = max(boxA[0], boxB[0])
    yA = max(boxA[1], boxB[1])
    xB = min(boxA[2], boxB[2])
    yB = min(boxA[3], boxB[3])
    inter_area = max(0, xB - xA) * max(0, yB - yA)
    boxA_area = (boxA[2] - boxA[0]) * (boxA[3] - boxA[1])
    boxB_area = (boxB[2] - boxB[0]) * (boxB[3] - boxB[1])
    union_area = boxA_area + boxB_area - inter_area
    return inter_area / (union_area + 1e-6)

ref_box  = [1.0, 1.0, 4.0, 4.0]
ann1_box = [1.1, 0.9, 4.1, 4.2]   # close
ann2_box = [0.5, 0.5, 3.5, 3.8]   # less precise

print(f"IoU (ann1 vs ref):  {compute_iou(ann1_box, ref_box):.3f}")
print(f"IoU (ann2 vs ref):  {compute_iou(ann2_box, ref_box):.3f}")
print(f"IoU (ann1 vs ann2): {compute_iou(ann1_box, ann2_box):.3f}")

fig, ax = plt.subplots(figsize=(5, 5))
ax.set_xlim(0, 5); ax.set_ylim(0, 5); ax.set_aspect('equal')
ax.add_patch(patches.Rectangle(
    (ref_box[0], ref_box[1]), ref_box[2]-ref_box[0], ref_box[3]-ref_box[1],
    linewidth=2.5, edgecolor='#2b3a8f', facecolor='none', label='Reference'))
ax.add_patch(patches.Rectangle(
    (ann1_box[0], ann1_box[1]), ann1_box[2]-ann1_box[0], ann1_box[3]-ann1_box[1],
    linewidth=2, edgecolor='#0d9e8e', facecolor='none', linestyle='--',
    label=f'Ann1 (IoU={compute_iou(ann1_box, ref_box):.2f})'))
ax.add_patch(patches.Rectangle(
    (ann2_box[0], ann2_box[1]), ann2_box[2]-ann2_box[0], ann2_box[3]-ann2_box[1],
    linewidth=2, edgecolor='#e05c5c', facecolor='none', linestyle=':',
    label=f'Ann2 (IoU={compute_iou(ann2_box, ref_box):.2f})'))
ax.legend(fontsize=10)
ax.set_title("Bounding Box Agreement (IoU)", fontsize=12)
plt.tight_layout()
plt.savefig('bbox_iou.png', dpi=150)
plt.show()
```

Semantic and Instance Segmentation

Segmentation annotation requires assigning a class label to every pixel in an image — among the most expensive annotation types.

Semantic segmentation: Each pixel belongs to a semantic class (road, sky, person, car). All pixels of the same class share the same label, regardless of which individual object they belong to.

Instance segmentation: Each individual object instance gets a unique label. A crowd of 20 people becomes 20 distinct masks.

Panoptic segmentation: Combines both: "thing" classes (countable objects) have instance masks; "stuff" classes (road, sky) have semantic masks.

SAM-assisted annotation: Meta's Segment Anything Model {cite}`kirillov2023segment` generates high-quality segmentation masks from a single point click. Annotators click the center of an object; SAM proposes a mask; the annotator accepts or corrects. The SAM authors report annotation engine speedups of roughly 6.5× over polygon-based labelling; gains vary across scene types and annotation tools.

SAM represents a broader shift: **the annotator's role changes from drawing to reviewing**. This has implications for annotation quality beyond speed. When annotators draw, their attention is engaged throughout the process. When annotators review and click "accept," there is evidence that they miss errors more readily — a version of automation bias specific to the annotation context.

Active Learning for Computer Vision

Active learning is particularly valuable in CV because: 1. Images are high-dimensional and feature-rich — embeddings from pre-trained models carry strong signals for uncertainty estimation 2. Large unlabeled pools are cheap (photos, video frames) 3. Annotation (especially segmentation) is expensive and cannot be easily crowdsourced for specialized domains

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)

X, y = make_classification(n_samples=3000, n_features=50, n_informative=25,
                           n_classes=5, n_clusters_per_class=2, random_state=42)
X_train, y_train = X[:2500], y[:2500]
X_test,  y_test  = X[2500:], y[2500:]

def margin_uncertainty(model, X_pool):
    probs = model.predict_proba(X_pool)
    sorted_p = np.sort(probs, axis=1)
    return sorted_p[:, -2] - sorted_p[:, -1]  # most negative = most uncertain

n_init = 50
results = {'active': [], 'random': []}
label_counts = list(range(50, 401, 30))

for strategy in ['active', 'random']:
```

```
    labeled = list(rng.choice(len(X_train), n_init, replace=False))
    unlabeled = [i for i in range(len(X_train)) if i not in labeled]

    for target in label_counts:
        while len(labeled) < target and unlabeled:
            if strategy == 'active' and len(labeled) >= 10:
                model_temp = LogisticRegression(max_iter=300).fit(
                    X_train[labeled], y_train[labeled])
                margins = margin_uncertainty(model_temp, X_train[unlabeled])
                idx = int(np.argmin(margins))
            else:
                idx = rng.integers(0, len(unlabeled))
            labeled.append(unlabeled.pop(idx))

        clf = LogisticRegression(max_iter=300).fit(X_train[labeled], y_train[labeled])
        results[strategy].append(accuracy_score(y_test, clf.predict(X_test)))

plt.figure(figsize=(7, 4))
plt.plot(label_counts, results['active'], 'o-', color='#2b3a8f',
         linewidth=2, label='Active (margin sampling)')
plt.plot(label_counts, results['random'], 's--', color='#e05c5c',
         linewidth=2, label='Random baseline')
plt.xlabel("Labeled training images", fontsize=12)
plt.ylabel("Test accuracy", fontsize=12)
plt.title("Active Learning for 5-Class Image Classification", fontsize=13)
plt.legend(fontsize=11); plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('cv_active_learning.png', dpi=150)
plt.show()
```

Semi-Supervised Learning with Human Guidance

The large quantity of unlabeled visual data available makes semi-supervised learning particularly attractive for CV.

Self-training / pseudo-labeling: Train a model on labeled data; use high-confidence predictions on unlabeled data as pseudo-labels; retrain. The critical design question is the confidence threshold. A low threshold brings in more examples but introduces noise; a high threshold leaves most of the unlabeled pool unused. Human involvement can guide this threshold — annotators spot-check a sample of pseudo-labeled examples to calibrate it.

FixMatch and consistency regularization: These methods train models to produce consistent predictions under augmentation. The key HITL insight: humans are consulted not just for labels but for **augmentation design** — which invariances should the model learn? A model for medical imaging should be invariant to rotation and brightness but not to scale; a model for text detection should not be made invariant to perspective distortion. These domain-specific choices require human expertise, and getting them wrong degrades semi-supervised learning substantially.

Active semi-supervised learning: The most efficient combination: active learning concentrates human labels where model uncertainty is highest; self-training auto-labels the high-confidence

tail. The human effort concentrates where it is most valuable, and the model bootstraps on the rest. At each cycle, a human audit of a random sample of pseudo-labels provides a quality check without requiring full review.

Video Annotation

Video annotation multiplies the challenges of image annotation by time:

- **Tracking:** Objects must be identified across frames. Annotators label keyframes; tracking algorithms interpolate between them. Tracking failures — occlusion, re-entry, rapid motion — require human re-labeling at higher rates than steady-state tracking.
- **Temporal consistency:** Boundaries drawn in frame t should be spatially consistent with frame $t + 1$. Inconsistent annotations are a training signal telling the model that objects jump discontinuously — a form of annotation noise that is particularly damaging for detection models.
- **Scalability:** A 1-hour video at 30fps is 108,000 frames. Full annotation is impractical; sampling strategies must be designed carefully to ensure that rare events (edge cases, near-misses, failure scenarios) are not systematically excluded.

Modern video annotation tools support **smart tracking** that propagates annotations across frames and flags frames where tracking confidence drops below a threshold, prompting the annotator to re-check. This is a direct application of the active learning idea to the annotation pipeline itself: the tool queries the annotator exactly where its interpolation is uncertain.

The rare event problem in autonomous systems. For applications where the consequences of rare events are catastrophic — autonomous driving, UAV navigation — the distribution of frames seen in normal operation is badly mismatched with the distribution of frames that matter most. A dataset built by uniformly sampling driving footage will contain millions of "nothing interesting is happening" frames and a handful of the near-accident, unusual-lighting, and degraded-sensor frames that actually matter for safety. HITL active learning that identifies and prioritizes such frames is not an efficiency hack — it is a safety requirement.

> **See also:** ImageNet dataset: {cite}`russakovsky2015imagenet`. Label bias in ImageNet: {cite}`yang2020towards`. COCO benchmark: {cite}`lin2014microsoft`. SAM (Segment Anything): {cite}`kirillov2023segment`. Core-set active learning for CV: {cite}`sener2018active`.

HITL in Healthcare and Science

Healthcare and science represent two of the domains where HITL ML is most consequential and most debated. The stakes are high: a missed cancer diagnosis or a flawed drug target has real human cost. Annotation requires rare and expensive expertise. Regulatory requirements constrain what models can do and how they must be validated. And unlike NLP, where the annotation problem is partially one of social construction, here there is often genuine ground truth — a tumor is present or it is not — even if no individual observer can reliably determine it.

The dominant framing in popular coverage is "AI vs. human": will AI replace radiologists? This framing is wrong in a way that matters. The real question is what form of human-AI collaboration produces better outcomes than either alone, and how to build systems that enable that collaboration rather than disrupting it.

Medical Image Analysis

Medical imaging — radiology (X-ray, CT, MRI), pathology (tissue slides), dermatology, ophthalmology — is the domain where medical AI has advanced most rapidly.

Expert Annotator Requirements

Medical image annotation typically requires physicians with specific subspecialty training. This makes annotation:

- **Slow:** Specialists have limited time; annotation competes with clinical duties
- **Expensive:** Costs range from tens to hundreds of dollars per annotated case, depending on subspecialty, modality, and task complexity
- **Variable:** Even specialists disagree, especially on borderline cases — a fact that is often treated as a problem but is actually informative

Inter-Radiologist Variability

Reader variability is well-documented in radiology. For chest X-ray interpretation, inter-reader disagreement is substantial — in the CheXNet study, four radiologists labelled the same pneumonia detection test set with F1 scores spanning a range of several percentage points {cite}`rajpurkar2017chexnet`, reflecting genuine diagnostic uncertainty on borderline cases. For nodule detection on lung CT, intra-reader variability (same reader, same case, different day) can be as large as inter-reader variability.

This variability is not just noise — it reflects genuine diagnostic uncertainty. Models trained on a single radiologist's annotations may learn that physician's specific biases rather than the underlying pathology.

:::{admonition} The CheXNet Controversy as a HITL Lesson :class: note

When Rajpurkar et al. claimed that their CheXNet model "exceeded radiologist performance" on pneumonia detection, the claim was immediately contested by the radiology community {cite}`yu2022assessing`. Part of the controversy was about the specific test set and radiologist comparison. But a deeper issue was methodological: the "radiologist performance" baseline used solo readers under time pressure, while clinical radiology typically involves consultation, comparison to prior imaging, and access to clinical context — none of which the model had.

The lesson is not that the model was good or bad, but that **performance comparisons require specifying the HITL setup**. A model that outperforms a solo radiologist reading cold may still be less accurate than a radiologist using the model's output as a second opinion. These are different systems with different error modes, and aggregating them differently. :::

:::{admonition} Soft labels in medicine :class: important

Several medical AI projects have moved to using **soft labels** that reflect the distribution of expert opinions rather than a single "gold standard" label. A chest X-ray labeled as 60% pneumonia / 40% atelectasis by a panel of radiologists carries more information than a forced binary choice. Models trained on such distributions show better calibration and more appropriate uncertainty quantification — and that uncertainty is clinically meaningful, since it tells the clinician when to consult, not just what the model thinks. :::

Active Learning for Rare Conditions

Active learning is especially valuable for rare diseases and rare pathologies, where even a large unlabeled pool contains only a small number of positive cases. Standard random sampling would waste expert time labeling mostly negative cases.

Uncertainty-based active learning naturally selects the borderline cases where the model is uncertain — which, for rare conditions, tend to be the positive cases and borderline negatives. This concentrates specialist time where it is most valuable. The combination of class-imbalanced training (with `class_weight='balanced'` or similar) and uncertainty-based selection is standard practice for rare-pathology detection tasks.

Clinical NLP Annotation

Electronic health records (EHRs) contain an enormous wealth of clinical narrative text: physician notes, discharge summaries, radiology reports, pathology reports. Extracting structured information from this text requires NLP — and high-quality NLP requires annotated training data.

Common clinical NLP annotation tasks: - **Clinical NER:** Identifying medications, dosages, diagnoses, procedures, and symptoms in text - **Negation detection:** "No evidence of pneumonia" vs. "Pneumonia confirmed" — a critical distinction that is surprisingly difficult - **Temporal reasoning:** Distinguishing current conditions from history ("history of MI, presented with chest pain") - **De-identification:** Removing Protected Health Information (PHI) to enable data sharing

PHI de-identification is both an annotation task and a data governance requirement. Under HIPAA (US) and GDPR (EU), health data cannot be shared without removing or anonymizing patient identifiers. Automated de-identification tools exist but are imperfect; human review

of automated outputs is standard practice, and the risk profile is asymmetric: false negatives (missed PHI) create legal exposure, making conservative thresholds necessary.

i2b2 / n2c2 as a Template

The i2b2 (Informatics for Integrating Biology and the Bedside) and successor n2c2 (National NLP Clinical Challenges) shared-task initiatives have released a series of expert-annotated clinical NLP datasets. These illustrate both the potential and the cost: annotation efforts typically involve teams of clinical domain experts working over several months, annotating hundreds of documents per challenge. The n2c2 datasets have catalyzed rapid progress precisely because they solved the data-sharing governance problem (de-identification + institutional agreements), not just the annotation problem.

Regulatory Considerations

Medical AI is subject to regulatory oversight in most jurisdictions.

FDA (United States): AI/ML-based Software as a Medical Device (SaMD) requires premarket approval or clearance. The FDA's 2021 AI/ML Action Plan emphasizes **predetermined change control plans** — documenting how the model will be updated and how those updates will be validated before deployment. A model that continuously learns from clinical feedback is, under this framework, a different device after each update and may require re-validation.

CE marking (Europe): Medical devices including AI systems must comply with the Medical Device Regulation (MDR). MDR requires clinical evaluation, post-market surveillance, and documentation of the data used for training and validation.

Key HITL implication: Regulatory frameworks require clear documentation of annotation processes, annotator qualifications, inter-rater reliability, and any changes to training data. This is not bureaucratic overhead — it is the audit trail that allows a clinician to understand what training data produced the model's current behavior, and it is legally required. HITL pipelines that treat annotation as an informal subprocess create regulatory risk that typically becomes visible only at the worst moment.

Scientific Data Annotation

Beyond healthcare, HITL ML plays a growing and underappreciated role in scientific research, where the annotation challenge often blends domain expertise with scale.

Astronomy: Galaxy Zoo

Galaxy Zoo {cite}`lintott2008galaxy` crowdsourced the morphological classification of galaxies from the Sloan Digital Sky Survey to citizen scientists. The original project collected over 40 million classifications from more than 100,000 volunteers, demonstrating that large-scale crowdsourcing of scientific image classification is feasible when the task can be decomposed into simple questions answerable without specialist training ("Is this galaxy smooth or featured?").

The Galaxy Zoo experience produced two important methodological findings. First, agreement between citizen scientists and professional astronomers was high for clear cases and

systematically diverged on edge cases — precisely the cases where the distinction matters scientifically. The solution was not to discard citizen science data on edge cases but to treat the distribution of volunteer responses as a soft label encoding genuine morphological ambiguity. Second, the classifier trained on Galaxy Zoo labels outperformed models trained to reproduce any single expert's labels, because the crowd distribution captured real visual uncertainty that a single expert's forced choice collapsed.

Genomics: Pathogenicity Classification

Annotating genomic variants — deciding whether a variant is pathogenic, benign, or of uncertain significance — is a high-stakes NLP and expert judgment problem. Clinical variant databases like ClinVar aggregate expert interpretations from multiple submitting labs, and disagreement between labs is common. Active learning is used to prioritize which variants require full expert review (literature search, functional evidence evaluation) versus which can be auto-classified by existing evidence. The result is a hybrid pipeline where most variants are handled by automated logic, a subset requires expert review, and the hardest cases are flagged for multi-lab consensus.

Climate and Earth Science

Labeling satellite imagery for land use change, deforestation, glacier extent, and storm tracks involves remote sensing experts and increasingly, citizen science platforms. The primary HITL challenge in this domain is temporal: labels made today may become stale as the world changes, and ground-truth verification (field surveys) is expensive and logistically constrained. Active learning that prioritizes images where the model's prediction disagrees with physical priors (e.g., predicting deforestation in a region known to be protected) is a practical way to direct scarce field verification resources.

Neuroscience: Connectomics

Reconstructing neural circuits from electron microscope images — connectomics — requires pixel-level annotation of neuron membranes across enormous image stacks. The Eyewire project gamified this task, engaging tens of thousands of players in tracing neurons through 3D image volumes. The gamification design solved a specific HITL problem: the task requires sustained attention and spatial reasoning over long sessions, which causes quality degradation in traditional annotation. Breaking the task into game segments with social mechanics maintained annotator engagement and quality at scales that professional annotation cannot achieve.

Managing Specialist Annotators

When annotation requires rare expertise, the usual crowdsourcing approaches (Chapter 13) do not apply.

The fundamental tension is that the people who can produce the highest-quality annotations are also the people whose time is most valuable and most constrained. Every design decision in a specialist annotation pipeline should be evaluated against the question: does this make the best use of scarce expert time?

What this means in practice:

- **Pre-annotate aggressively.** Use lower-tier annotators, automated models, or rule-based systems to generate candidates that the specialist reviews and corrects rather than creating from scratch. The specialist's judgment is the bottleneck; feeding them a pre-annotation to correct is faster than asking them to annotate from a blank screen, provided the pre-annotation quality is sufficient that correction is not slower than starting over.
- **Design for expert attention, not throughput.** Annotation interfaces optimized for high throughput (rapid binary decisions, keyboard shortcuts, minimal display) are appropriate for crowdsourcing. Specialist annotation often benefits from richer interfaces: side-by-side comparison with prior cases, easy access to reference materials, annotation confidence fields, and the ability to flag a case for discussion. These slow individual annotations but improve quality and reduce the need for re-annotation.
- **Track individual annotator patterns explicitly.** With a small pool of specialists, it is feasible and important to track each annotator's agreement rate with the panel, flag cases that seem inconsistent with their own history, and discuss them in regular calibration sessions. This is not surveillance — it is the same quality process that clinical medicine uses for performance review, and specialists generally respond well when it is framed as shared quality improvement rather than assessment.
- **Session design matters.** Medical annotation is cognitively demanding. Evidence from radiology and pathology suggests that error rates increase measurably after roughly 90 minutes of continuous reading, and that breaks of even a few minutes can partially restore attention. Annotation interfaces that enforce break prompts (without making them dismissable) are a simple HITL design decision with real quality impact.

A HITL Active Learning Pipeline for Medical Imaging

```python
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
from sklearn.datasets import make_classification
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)

# Simulate a rare-pathology detection task
# 8% positive class (e.g., rare pathology)
X, y = make_classification(
    n_samples=5000, n_features=100,
    n_informative=20, n_redundant=10,
    weights=[0.92, 0.08],
    random_state=42
)
X_train, y_train = X[:4000], y[:4000]
X_test,  y_test  = X[4000:], y[4000:]

print(f"Training set positive prevalence: {y_train.mean():.1%}")

def run_medical_al(strategy, n_initial=50, budget=300):
    labeled = list(rng.choice(len(X_train), n_initial, replace=False))
```

```
    unlabeled = [i for i in range(len(X_train)) if i not in labeled]
    aucs = []

    while len(labeled) < n_initial + budget:
        model = LogisticRegression(max_iter=500, class_weight='balanced')
        model.fit(X_train[labeled], y_train[labeled])

        if len(labeled) % 30 == 0:
            preds = model.predict_proba(X_test)[:, 1]
            aucs.append(roc_auc_score(y_test, preds))

        X_pool = X_train[unlabeled]
        if strategy == 'uncertainty' and len(labeled) >= 10:
            probs = model.predict_proba(X_pool)
            entropy = -np.sum(probs * np.log(probs + 1e-9), axis=1)
            q = int(np.argmax(entropy))
        else:
            q = rng.integers(0, len(unlabeled))

        labeled.append(unlabeled.pop(q))

    return np.array(aucs)

aucs_al  = run_medical_al('uncertainty')
aucs_rnd = run_medical_al('random')
label_counts = np.arange(len(aucs_al)) * 30 + 50

plt.figure(figsize=(7, 4))
plt.plot(label_counts, aucs_al,  'o-',  color='#2b3a8f', linewidth=2, label='Uncertainty AL')
plt.plot(label_counts, aucs_rnd, 's--', color='#e05c5c', linewidth=2, label='Random baseline'
plt.xlabel("Expert labels obtained", fontsize=12)
plt.ylabel("AUROC", fontsize=12)
plt.title("Active Learning for Rare Pathology Detection", fontsize=13)
plt.legend(); plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('medical_al.png', dpi=150)
plt.show()

# Show how AL preferentially selects positive examples
# by checking which class the queried examples belong to
n_init = 50
labeled_al  = list(rng.choice(len(X_train), n_init, replace=False))
labeled_rnd = labeled_al.copy()
unlabeled_al  = [i for i in range(len(X_train)) if i not in labeled_al]
unlabeled_rnd = unlabeled_al.copy()

model = LogisticRegression(max_iter=500, class_weight='balanced')
model.fit(X_train[labeled_al], y_train[labeled_al])
probs = model.predict_proba(X_train[unlabeled_al])
entropy = -np.sum(probs * np.log(probs + 1e-9), axis=1)
top50_al  = [unlabeled_al[i] for i in np.argsort(entropy)[-50:]]
```

```
top50_rnd = list(rng.choice(unlabeled_rnd, 50, replace=False))

pos_rate_al  = y_train[top50_al].mean()
pos_rate_rnd = y_train[top50_rnd].mean()
print(f"\nPositive rate in next 50 queries:")
print(f"  Uncertainty AL: {pos_rate_al:.1%}  (base rate: {y_train.mean():.1%})")
print(f"  Random:         {pos_rate_rnd:.1%}")
print(f"  AL queries {pos_rate_al/y_train.mean():.1f}x more positives than random")
```

See also: Galaxy Zoo crowdsourcing: {cite}`lintott2008galaxy`. CheXNet radiologist performance: {cite}`rajpurkar2017chexnet`. Radiograph quality and AI-assisted diagnosis: {cite}`yu2022assessing`. Clinical NLP annotation methodology: {cite}`pustejovsky2012natural`. For FDA AI/ML action plan guidance, see the FDA's published documentation (2021).

Annotation Platforms and Tooling

The annotation platform is the environment where human feedback becomes data. A good platform increases throughput, reduces errors, maintains quality controls, and makes the annotation pipeline manageable at scale. Choosing the right platform — and knowing when to build versus buy — is a consequential decision in any HITL project.

The Annotation Platform Landscape

The market for annotation tooling has grown and matured substantially in recent years, driven by enterprise demand for ML training data. Tools span a wide range from fully managed services to open-source self-hosted frameworks.

Open-Source Platforms

Label Studio is the most popular open-source annotation platform, supporting text, images, audio, video, and time-series data through a unified XML-based task configuration. It can be self-hosted and integrates with ML backends for active learning. Key strengths: flexibility, community support, and the ability to embed custom ML predictions for pre-annotation.

Prodigy (from the makers of spaCy) is a highly workflow-oriented annotation tool designed for NLP tasks. Its streaming architecture sends examples one at a time and supports active learning out of the box. The annotation interface is minimal but fast — designed to maximize annotation throughput. Prodigy is paid software but widely used in NLP research.

CVAT (Computer Vision Annotation Tool) is the leading open-source tool for CV annotation, with strong support for detection, segmentation, and video annotation. Originally developed at Intel, CVAT supports tracking algorithms, skeleton annotation, and third-party algorithm integrations.

Doccano targets sequence labeling tasks (NER, relation extraction, text classification). Its simple web interface makes it accessible for teams without dedicated data engineering resources.

Commercial Platforms

Scale AI provides end-to-end managed annotation services: human workforce, quality management, and API integration. Particularly strong for autonomous driving, robotics, and complex 3D annotation. Pricing is based on task complexity and volume.

Labelbox is a full platform for data curation, annotation, and ML-assisted labeling. Strong enterprise features: project management, quality workflows, model feedback loops, and integrations with major ML platforms (SageMaker, Vertex AI, Azure ML).

Appen (formerly Figure Eight / CrowdFlower) operates a large global annotation workforce alongside tooling. A good choice when volume and workforce management are primary concerns.

Surge AI focuses on expert annotators and is strong for tasks requiring domain knowledge or nuanced judgment.

Humanloop specializes in LLM feedback collection — preference annotation, RLHF data collection, and model evaluation.

Annotation Platform Feature Comparison

Feature	Label Studio	Prodigy	CVAT	Labelbox	Scale AI
License	Open source	Commercial	Open source	Commercial	Commercial
Hosting	Self / cloud	Self	Self / cloud	Cloud	Managed
Text annotation	□	□	—	□	□
Image annotation	□	Limited	□	□	□
Video annotation	□	—	□	□	□
Active learning integration	□	□	Limited	□	□
ML-assisted pre-annotation	□	□	□	□	□
Quality control workflows	Basic	Basic	Basic	Advanced	Advanced
API / Programmatic access	□	□	□	□	□
Workforce management	—	—	—	Limited	□

Annotation as Code

A critical but often overlooked aspect of annotation infrastructure is **version control for annotations and annotation schemas**. Treating annotation as code means:

Schema-first design. The label taxonomy and annotation rules are defined in a versioned configuration file (YAML or JSON) before annotation begins. Changes to the schema create a new version.

Annotation versioning. Annotations are stored with a link to the schema version under which they were created. This allows auditing, rolling back, and comparing annotations across schema versions.

Reproducible pipelines. The annotation pipeline — from raw data to training-ready labels — should be reproducible from code. Pre-annotation models, quality filters, aggregation logic, and data splits should all be recorded.

```
# Example: Label Studio annotation schema (text classification)
label_config: |
  <View>
    <Text name="text" value="$text"/>
    <Choices name="sentiment" toName="text" choice="single">
      <Choice value="positive"/>
      <Choice value="negative"/>
      <Choice value="neutral"/>
      <Choice value="mixed"/>
    </Choices>
  </View>
schema_version: "2.1.0"
task_type: text_classification
guidelines_version: "guidelines_v3.pdf"
```text
Data source
    |
    v
Unlabeled pool -->  Annotation platform --> Labeled dataset
    ^                    |                        |
    |                    | (ML-assisted)           v
Active learning <--------+                    Training run
query strategy                                     |
    ^                                              v
    +------------ Trained model <---------- Evaluation
                                                   |
                                            Deployment &
                                            monitoring
```
```

Key integration points: 1. **Data ingestion:** Unlabeled data flows automatically from the data warehouse to the annotation platform 2. **Model serving:** The current best model is deployed to the annotation platform for pre-annotation and active learning scoring 3. **Export:** Completed annotations are exported in a format compatible with the training framework (COCO JSON, Hugging Face datasets, etc.) 4. **Feedback loop:** Production model errors are routed back to the annotation platform for correction

> **See also:** For Label Studio documentation and active learning integration: https://labelstud.io. For Prodigy: https://prodi.gy. CVAT: https://cvat.ai. For a comprehensive comparison of annotation tools, see {cite}`monarch2021human`, Chapter 7.

Crowdsourcing and Quality Control

When annotation tasks are simple enough to be performed by non-experts, crowdsourcing platforms offer access to large, on-demand annotation workforces at low per-item cost. Building high-quality labeled datasets from crowds requires careful task design, strategic redundancy, and rigorous quality control.

Crowdsourcing Platforms

Amazon Mechanical Turk (MTurk) is the original crowdsourcing marketplace, launched in 2005. Workers ("Turkers") complete micro-tasks (HITs) posted by requesters. A 2018 study found median effective hourly earnings for Turkers of approximately $2/hr — well below minimum wage in many high-income countries {cite}`hara2018data` — an ethical concern addressed later in Chapter 15. MTurk is best for simple tasks with clear, verifiable criteria.

Prolific is an academic crowdsourcing platform that enforces a minimum pay standard (currently around £9/hr, roughly $11/hr, as stated in Prolific's published guidelines), screens participants by demographics, and maintains a panel of workers who have opted in to research participation. Preferred for social science research and tasks requiring representativeness.

Appen (and similar: Telus International, iMerit) provides managed annotation workforces with quality management, used for higher-complexity tasks and enterprise projects.

Specialized communities. For domain-specific tasks, communities of domain enthusiasts can provide high-quality annotations: Galaxy Zoo for astronomy, eBird for bird species, Chess Tempo for chess position annotation.

Task Design for Crowdsourcing

Decompose Complex Tasks

Complex tasks should be decomposed into simple, well-defined micro-tasks. Instead of asking workers to comprehensively annotate a document, ask them one focused question at a time: "Does this sentence contain a person's name?" or "Rate the clarity of this translation on a scale of 1–5."

Benefits of decomposition: - Lower cognitive demand per task □ less fatigue, higher quality - Each micro-task can be separately quality-controlled - Easier to audit and debug

The Importance of Instructions

The single biggest predictor of crowdsourcing quality is instruction quality. Good task instructions: - Explain the *purpose* of the task in one sentence - Give a clear, unambiguous definition of each category - Provide 3–5 worked examples (especially edge cases) - Are no longer than workers will actually read (< 300 words for simple tasks)

Run a **pilot study** (10–50 workers, 20–100 tasks) before scaling. Analyze pilot disagreements; most point to instruction ambiguities that can be fixed.

Gold Standard Questions

Embed **gold standard questions** — tasks with known correct answers — throughout the task batch. Workers who fail gold questions below a threshold are removed from the project.

```
import numpy as np
from scipy.stats import binom

rng = np.random.default_rng(42)

def simulate_gold_screening(n_workers=100, gold_per_batch=5,
                            p_good_worker=0.7, good_acc=0.92,
                            bad_acc=0.55, threshold=0.70):
    """
    Simulate quality screening via gold questions.
    Returns: precision and recall of identifying bad workers.
    """
    worker_quality = rng.choice(['good', 'bad'], size=n_workers,
                                p=[p_good_worker, 1 - p_good_worker])
    results = {'tp': 0, 'fp': 0, 'tn': 0, 'fn': 0}

    for q in worker_quality:
        acc = good_acc if q == 'good' else bad_acc
        n_correct = rng.binomial(gold_per_batch, acc)
        passed = (n_correct / gold_per_batch) >= threshold
        if q == 'bad' and not passed:  results['tp'] += 1
        if q == 'good' and not passed: results['fp'] += 1
        if q == 'bad' and passed:      results['fn'] += 1
        if q == 'good' and passed:     results['tn'] += 1

    tp, fp, fn = results['tp'], results['fp'], results['fn']
    precision = tp / (tp + fp + 1e-6)
    recall    = tp / (tp + fn + 1e-6)
    return precision, recall

# Vary gold question count
gold_counts = [3, 5, 8, 10, 15, 20]
precisions, recalls = [], []
for g in gold_counts:
    p_list, r_list = [], []
    for _ in range(50):
        p, r = simulate_gold_screening(gold_per_batch=g)
        p_list.append(p); r_list.append(r)
```

```
    precisions.append(np.mean(p_list))
    recalls.append(np.mean(r_list))

import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(gold_counts, precisions, 'o-', color='#2b3a8f', label='Precision', linewidth=2)
ax.plot(gold_counts, recalls,    's--', color='#0d9e8e', label='Recall',    linewidth=2)
ax.set_xlabel("Gold questions per batch", fontsize=12)
ax.set_ylabel("Screening performance", fontsize=12)
ax.set_title("Worker Screening via Gold Standard Questions", fontsize=13)
ax.legend(fontsize=11); ax.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('gold_screening.png', dpi=150)
plt.show()

p, r = simulate_gold_screening(gold_per_batch=5)
print(f"5 gold questions: Precision={p:.3f}, Recall={r:.3f}")
p, r = simulate_gold_screening(gold_per_batch=10)
print(f"10 gold questions: Precision={p:.3f}, Recall={r:.3f}")
```

Statistical Models for Label Aggregation

Majority vote is a natural baseline but ignores differences in annotator accuracy. Statistical models can do better.

The Dawid-Skene Model

The **Dawid-Skene (DS) model** {cite}`dawid1979maximum` jointly estimates: - The **true label** z_i for each item i - The **confusion matrix** $\pi_j^{(k,l)}$ for each annotator j: the probability that annotator j labels an item with true class k as class l

The EM algorithm iterates: - **E-step:** Given annotator confusion matrices, compute posterior probability of each true label - **M-step:** Given item label estimates, update annotator confusion matrices

```
import numpy as np
from scipy.special import softmax

def dawid_skene_em(annotations, n_classes, n_iter=20):
    """
    Simplified Dawid-Skene EM for binary classification.
    annotations: dict {item_idx: [(annotator_idx, label), ...]}
    Returns: estimated true labels and annotator accuracies.
    """
    items = sorted(annotations.keys())
    n_items = len(items)
    annotators = sorted({a for anns in annotations.values() for a, _ in anns})
    n_annotators = len(annotators)
    ann_idx = {a: i for i, a in enumerate(annotators)}
```

```
    # Initialize: majority vote
    T = np.zeros((n_items, n_classes))  # soft label estimates
    for i, item in enumerate(items):
        for _, label in annotations[item]:
            T[i, label] += 1
        T[i] /= T[i].sum()

    # Confusion matrices: shape (n_annotators, n_classes, n_classes)
    PI = np.ones((n_annotators, n_classes, n_classes)) * 0.5

    for _ in range(n_iter):
        # M-step: update confusion matrices
        PI = np.zeros((n_annotators, n_classes, n_classes)) + 1e-6
        for i, item in enumerate(items):
            for ann, label in annotations[item]:
                a = ann_idx[ann]
                for k in range(n_classes):
                    PI[a, k, label] += T[i, k]
        PI /= PI.sum(axis=2, keepdims=True)

        # E-step: update soft label estimates
        T = np.zeros((n_items, n_classes))
        for i, item in enumerate(items):
            log_p = np.zeros(n_classes)
            for ann, label in annotations[item]:
                a = ann_idx[ann]
                log_p += np.log(PI[a, :, label] + 1e-10)
            T[i] = softmax(log_p)

    return {item: T[i] for i, item in enumerate(items)}, PI

# Simulate crowdsourcing scenario
rng = np.random.default_rng(42)
N_ITEMS, N_ANNOTATORS, ANNS_PER_ITEM = 200, 10, 3
true_labels = rng.integers(0, 2, N_ITEMS)
# Annotator accuracies: 3 good (0.9), 4 medium (0.75), 3 noisy (0.6)
accuracies = [0.90]*3 + [0.75]*4 + [0.60]*3

annotations = {}
for i in range(N_ITEMS):
    anns_for_item = []
    chosen = rng.choice(N_ANNOTATORS, ANNS_PER_ITEM, replace=False)
    for a in chosen:
        acc = accuracies[a]
        label = true_labels[i] if rng.random() < acc else 1 - true_labels[i]
        anns_for_item.append((a, int(label)))
    annotations[i] = anns_for_item

# Run Dawid-Skene
soft_labels, confusion = dawid_skene_em(annotations, n_classes=2, n_iter=30)
ds_preds = {i: int(np.argmax(soft_labels[i])) for i in range(N_ITEMS)}
```

```
# Compare with majority vote
mv_preds = {}
for i in range(N_ITEMS):
    votes = [l for _, l in annotations[i]]
    mv_preds[i] = int(np.round(np.mean(votes)))

ds_acc = np.mean([ds_preds[i] == true_labels[i] for i in range(N_ITEMS)])
mv_acc = np.mean([mv_preds[i] == true_labels[i] for i in range(N_ITEMS)])

print(f"Majority vote accuracy:  {mv_acc:.3f}")
print(f"Dawid-Skene accuracy:    {ds_acc:.3f}")
print(f"\nEstimated annotator accuracies (diagonal of confusion matrix):")
for a in range(N_ANNOTATORS):
    est_acc = np.mean([confusion[a, k, k] for k in range(2)])
    print(f"  Annotator {a}: estimated={est_acc:.3f}, true={accuracies[a]:.2f}")
```

MACE

MACE (Multi-Annotator Competence Estimation) {cite}`hovy2013learning` is an alternative probabilistic model that explicitly represents annotator spamming (random labeling) vs. competent annotation. An annotator either provides a meaningful label (with probability $1 - \text{spam}_j$) or a random label (with probability spam_j). This two-component model is often better calibrated than Dawid-Skene for crowdsourcing scenarios where some annotators are pure spammers.

Redundancy and Aggregation Strategy

The optimal number of annotators per item depends on the task difficulty and annotator quality:

- **Easy tasks with skilled annotators:** 1–2 annotators per item is often sufficient
- **Moderate tasks with trained annotators:** 3 annotators + majority vote
- **Difficult/subjective tasks with crowdworkers:** 5–7 annotators + Dawid-Skene

The key insight: redundancy is most valuable when annotator accuracy is low. For annotators with accuracy p, majority vote accuracy with n annotators is:

$$P(\text{MV correct}) = \sum_{k>n/2}^{n} \binom{n}{k} p^k (1-p)^{n-k}$$

For $p = 0.70$, adding a third annotator increases majority vote accuracy from 70% to 78%; for $p = 0.90$, the gain from a third annotator is negligible (from 90% to 97%).

> **See also:** Dawid-Skene model: {cite}`dawid1979maximum`. MACE: {cite}`hovy2013learning`. For a comprehensive review of crowdsourcing for NLP: {cite}`snow2008cheap`. Crowd ethics and fair pay: see Chapter 15.

Evaluation and Metrics

Knowing whether your HITL system is working requires more than measuring model accuracy. You need to know whether you are getting value from your annotation budget, whether the model is actually better aligned with human intent, and whether additional human feedback will continue to improve things. This chapter covers the full landscape of evaluation in HITL settings.

Model-Centric Metrics

Standard ML metrics apply directly to HITL systems, with some important nuances.

Classification Metrics

Accuracy is appropriate when classes are balanced and all errors are equally costly. In HITL settings, however, the labeled test set may be biased by the query strategy (active learning queries non-random examples), making simple accuracy estimates unreliable.

F1 score is the harmonic mean of precision and recall, appropriate for imbalanced classes. In HITL contexts, both precision and recall may matter differently depending on the cost asymmetry between false positives and false negatives.

AUROC measures the model's ability to discriminate between classes regardless of threshold — important for calibration-sensitive tasks like medical screening.

Calibration measures how well predicted probabilities correspond to empirical frequencies. In HITL systems, models trained on biased labeled sets (from active learning) can be miscalibrated even when accurate.

Generative Model Metrics

For language models and generative systems, evaluation is fundamentally harder. No single automatic metric captures quality:

- **BLEU / ROUGE / METEOR:** Reference-based metrics for translation and summarization. Correlate weakly with human quality judgments for long-form generation.
- **Perplexity:** Measures how well the model predicts held-out text. A necessary but not sufficient condition for quality.
- **BERTScore:** Embedding-based similarity to references. Better correlated with human judgments than n-gram metrics.
- **Human evaluation:** The gold standard. See Section 14.3.

Annotation Efficiency Metrics

HITL evaluation should also measure whether human feedback is being used efficiently.

Learning Curves

A **learning curve** plots model performance as a function of the number of labeled examples. A steep learning curve (rapid improvement with few labels) indicates that the annotation strategy is selecting informative examples. A flat learning curve indicates that additional labeling is providing diminishing returns.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score, roc_auc_score
from sklearn.model_selection import StratifiedShuffleSplit

rng = np.random.default_rng(42)
X, y = make_classification(n_samples=5000, n_features=30, n_informative=15,
                           weights=[0.8, 0.2], random_state=42)
X_test, y_test = X[4000:], y[4000:]
X_pool, y_pool = X[:4000], y[:4000]

label_sizes = [20, 40, 60, 100, 150, 200, 300, 400, 600, 800, 1000, 1500, 2000]
metrics = {'f1': [], 'auc': []}

for n in label_sizes:
    idx = rng.choice(len(X_pool), n, replace=False)
    clf = LogisticRegression(max_iter=500, class_weight='balanced')
    clf.fit(X_pool[idx], y_pool[idx])
    preds = clf.predict(X_test)
    probs = clf.predict_proba(X_test)[:, 1]
    metrics['f1'].append(f1_score(y_test, preds))
    metrics['auc'].append(roc_auc_score(y_test, probs))

# Fit learning curve: performance ≈ a - b/sqrt(n)
from scipy.optimize import curve_fit

def learning_curve_fn(n, a, b):
    return a - b / np.sqrt(n)

popt_f1, _ = curve_fit(learning_curve_fn, label_sizes, metrics['f1'], p0=[0.9, 2])
popt_auc, _ = curve_fit(learning_curve_fn, label_sizes, metrics['auc'], p0=[0.95, 1])

n_smooth = np.linspace(20, 3000, 200)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4))

ax1.scatter(label_sizes, metrics['f1'], color='#2b3a8f', zorder=5, s=40)
ax1.plot(n_smooth, learning_curve_fn(n_smooth, *popt_f1), '--', color='#e05c5c',
         label=f'Fit: {popt_f1[0]:.3f} - {popt_f1[1]:.1f}/√n')
```

```
ax1.set_xlabel("Labeled examples"); ax1.set_ylabel("F1 score")
ax1.set_title("Learning Curve: F1"); ax1.legend(); ax1.grid(alpha=0.3)

ax2.scatter(label_sizes, metrics['auc'], color='#0d9e8e', zorder=5, s=40)
ax2.plot(n_smooth, learning_curve_fn(n_smooth, *popt_auc), '--', color='#e05c5c',
         label=f'Fit: {popt_auc[0]:.3f} - {popt_auc[1]:.2f}/√n')
ax2.set_xlabel("Labeled examples"); ax2.set_ylabel("AUROC")
ax2.set_title("Learning Curve: AUROC"); ax2.legend(); ax2.grid(alpha=0.3)

plt.tight_layout()
plt.savefig('learning_curves.png', dpi=150)
plt.show()

# Estimate the annotation budget needed to reach a target performance
target_f1 = 0.80
n_needed = (popt_f1[1] / (popt_f1[0] - target_f1)) ** 2
print(f"Estimated labels needed to reach F1={target_f1}: {n_needed:.0f}")
```

Return on Investment (ROI) Analysis

The ROI of human feedback answers: for each additional label, how much does model performance improve?

$$\text{ROI}(n) = \frac{\Delta\text{performance}(n)}{\text{cost per label}}$$

As a model matures (and as the easy-to-learn examples are exhausted), ROI typically decreases. The practical implication: annotation budgets should be front-loaded, with more labels collected in early stages when ROI is highest.

Human Evaluation

For generative systems and subjective tasks, human evaluation remains the gold standard.

Direct Assessment (DA)

Annotators rate outputs on an absolute scale (e.g., 1–100 for translation quality, or 1–5 for response helpfulness). DA has been standardized in machine translation evaluation (WMT benchmarks).

Best practices for DA: - Randomize the order of outputs to prevent anchoring - Use a sufficient number of annotators per item (3–5 minimum) - Include quality controls (obviously good and bad examples to catch inattentive raters) - Report inter-annotator agreement alongside aggregate scores

Comparative Evaluation

Annotators choose between two outputs: "Which is better?" Comparative judgments are faster and more consistent than absolute ratings (see Chapter 8). **ELO rating systems** (borrowed from chess) convert pairwise comparison outcomes into a continuous quality ranking.

```python
import numpy as np

def update_elo(rating_a, rating_b, outcome_a, k=32):
    """Update ELO ratings. outcome_a: 1=A wins, 0=B wins, 0.5=tie."""
    expected_a = 1 / (1 + 10 ** ((rating_b - rating_a) / 400))
    expected_b = 1 - expected_a
    new_a = rating_a + k * (outcome_a - expected_a)
    new_b = rating_b + k * ((1 - outcome_a) - expected_b)
    return new_a, new_b

# Simulate 5 model versions being compared pairwise
rng = np.random.default_rng(42)
true_quality = [0.60, 0.70, 0.75, 0.80, 0.85]  # underlying model quality
n_models = len(true_quality)
elo_ratings = {i: 1000.0 for i in range(n_models)}

for _ in range(500):  # 500 pairwise comparisons
    i, j = rng.choice(n_models, 2, replace=False)
    p_i_wins = true_quality[i] / (true_quality[i] + true_quality[j])
    outcome = 1.0 if rng.random() < p_i_wins else 0.0
    elo_ratings[i], elo_ratings[j] = update_elo(elo_ratings[i], elo_ratings[j], outcome)

print("ELO Rankings after 500 comparisons:")
sorted_models = sorted(elo_ratings.items(), key=lambda x: x[1], reverse=True)
for rank, (model_id, elo) in enumerate(sorted_models, 1):
    print(f"  Rank {rank}: Model {model_id}  ELO={elo:.1f}  True quality={true_quality[model_
```

Behavioral Testing (CheckList)

CheckList {cite}`ribeiro2020beyond` is a methodology for systematic behavioral evaluation of NLP models. Instead of random test sets, it designs test cases that probe specific capabilities:

- **Minimum Functionality Tests (MFT):** Does the model handle simple, obvious cases?
- **Invariance Tests (INV):** Does the model's output change when it shouldn't (e.g., when paraphrasing)?
- **Directional Expectation Tests (DIR):** Does the model's output change in the expected direction when input changes?

CheckList makes human evaluation targeted and actionable: instead of a single accuracy number, it provides a capability profile.

Measuring Alignment with Human Intent

For RLHF systems, measuring alignment is a central evaluation challenge.

Reward model evaluation: The reward model's accuracy on a held-out preference test set. Ouyang et al. {cite}`ouyang2022training` report approximately 72% pairwise accuracy for InstructGPT's reward model; as a rough point of reference, figures in this vicinity are commonly cited for similar RLHF pipelines, though results vary widely by task and data quality.

Win rate: Given two model versions (e.g., SFT baseline vs. RLHF fine-tuned), what fraction of responses does the RLHF model win in human pairwise comparisons?

GPT-4 as evaluator: Using a capable LLM to evaluate responses has become common for rapid iteration. Gilardi et al. {cite}`gilardi2023chatgpt` and Zheng et al. {cite}`zheng2023judging` find LLM evaluator agreement with human judgment ranging roughly from 0.7 to 0.9 depending on task — useful for rapid A/B comparison, but less reliable for detecting sycophancy, cultural nuance, or safety issues.

Sycophancy detection: Measure whether the model changes its answers based on implied user preference (e.g., "I think X is correct; what do you think?"). A well-aligned model should not be sycophantic.

A/B Testing in Deployed Systems

For systems in production, the ultimate evaluation is **A/B testing**: route a fraction of users to the new model version and measure downstream outcomes.

A/B testing gives an unbiased estimate of model quality in the actual deployment context, capturing effects that laboratory evaluation misses (user behavior, population distribution, edge cases).

The challenge: appropriate downstream metrics. Engagement metrics (clicks, session length) may reward manipulative behavior. Task completion rates or user satisfaction surveys are better-aligned but noisier.

> **See also:** CheckList behavioral testing: {cite}`ribeiro2020beyond`. For RLHF evaluation methodology, see {cite}`ouyang2022training`. For human evaluation best practices in MT: {cite}`graham2015accurate`. For learning curve theory: {cite}`mukherjee2003estimating`.

Fairness, Bias, and Ethics

Human-in-the-loop systems inherit both the capabilities and the limitations of the humans in them. Annotators bring knowledge, judgment, and creativity to their work — but also biases, fatigue, and the social context of their lives. The choices we make in designing HITL systems — who annotates, what they are asked, how they are compensated, and how their work is used — have consequences that extend far beyond model accuracy metrics.

This chapter addresses the ethical dimensions of HITL ML.

Sources of Bias in HITL Systems

Annotator Demographics

Annotation is not a neutral act. The labels annotators assign reflect their perspectives, experiences, and cultural contexts. When the annotation workforce is demographically homogeneous — as it often is, with crowdwork skewing young, male, and Western — the resulting dataset encodes the perspectives of that group.

Evidence: Studies of NLP annotation datasets have found that annotators' demographic characteristics predict their label choices for subjective tasks (toxicity, sentiment, offensiveness). Datasets annotated by predominantly U.S.-based crowdworkers encode U.S. cultural norms that do not generalize to other regions {cite}`geva2019annotator`.

Consequences: Models trained on such data perform well for users who resemble the annotator pool and less well — or in biased ways — for users who don't.

Mitigation: Deliberate workforce diversification; stratified annotation (ensure annotators from relevant demographic groups contribute to relevant tasks); track annotator demographics and their effect on label distributions.

Task Framing

How a question is framed influences the answers it elicits. If annotators are asked "Is this text toxic?" they may answer differently than if asked "Would this text harm someone who belongs to the mentioned group?" The framing embeds assumptions about what matters.

Example: Annotation of "abusive language" on social media varies significantly depending on whether annotators are shown contextual information about the author's identity. A statement that appears threatening in isolation may be recognized as reclaimed language or in-group humor when context is provided.

Platform Effects

The platform and payment structure affect annotation quality. Workers who are paid per task rather than per hour have an incentive to work quickly; this increases throughput but decreases quality. Workers who fear being blocked from a platform for low accuracy may avoid marking "uncertain" and instead make guesses, masking genuine ambiguity.

Confirmation and Anchoring Biases

Annotators are influenced by: - **Pre-annotation:** Model predictions shown to annotators are accepted more often than rejected, even when wrong - **Order effects:** Labeling the same item in different contexts yields different answers - **Priming:** Previous items in a task affect how subsequent items are labeled

---

Fairness in HITL Systems

What Is Fairness?

Fairness in ML is a contested concept with multiple formal definitions that are often mutually incompatible {cite}`barocas2019fairness`. For HITL purposes, the most relevant dimensions are:

Representation: Is the training data representative of the populations the model will affect?

Performance parity: Does the model perform equally well across different demographic groups?

Labeling consistency: Are the same behaviors labeled identically regardless of who is performing them? (Research has shown this is not always the case — identical content is sometimes labeled differently when attributed to different racial or gender groups.)

Fairness-Aware Active Learning

Standard active learning queries focus on model uncertainty, which tends to concentrate on majority-class examples. This can exacerbate performance disparities for minority groups.

Fairness-aware query strategies augment the uncertainty criterion with diversity or representation constraints:

$$x^* = \arg\max_{x \in \mathcal{U}} \left[\lambda \cdot \text{uncertainty}(x) + (1 - \lambda) \cdot \text{minority_group_bonus}(x)\right]$$

Setting $\lambda < 1$ ensures the query strategy does not entirely ignore minority representation.

Annotator Welfare

The Ghost Work Problem

The annotation work that powers ML is largely invisible. Data workers — often in the Global South — label images, transcribe speech, and moderate content for low wages, without benefits, in gig-economy arrangements with no job security. Kate Crawford and Vladan Joler's

"Anatomy of an AI System" {cite}`crawford2018anatomy` and Mary Gray and Siddharth Suri's "Ghost Work" {cite}`gray2019ghost` documented the scale and precarity of this labor.

Amazon MTurk statistics: A 2018 systematic analysis of MTurk earnings found median effective hourly pay of approximately $2/hr — below minimum wage in most U.S. states and many high-income countries {cite}`hara2018data`. Workers outside high-income countries often face additional barriers: requesters frequently restrict high-paying tasks to US-only qualifications, and the pool of workers competing for the remaining open tasks is global, compressing effective earnings further.

Content moderation: A particularly harmful form of annotation work — reviewing graphic, violent, and disturbing content — has been linked to PTSD, depression, and anxiety among workers {cite}`newton2019trauma`. Platforms have faced criticism for inadequate mental health support and excessive exposure quotas.

Ethical Practices

Fair pay: Pay annotation workers at or above the local minimum wage. Research has shown that higher pay attracts higher-quality workers without proportionally increasing costs per correct label.

Work visibility: Acknowledge the labor that creates training data in publications and product documentation.

Mental health support: For workers who review harmful content, provide psychological support, rotation schedules, and exposure limits.

Worker representation: Enable annotation workers to report concerns, request guideline clarifications, and challenge unfair quality assessments.

Privacy in Annotation

Protected Health Information (PHI) and PII

Annotation tasks often involve sensitive personal data. A medical annotation project may expose workers to patient records; an NLP project may expose workers to private communications; a content moderation project exposes workers to users' personal disclosures.

Regulatory frameworks (HIPAA, GDPR) restrict how personal data can be shared with annotation workforces. Key principles:

- **Data minimization:** Share only the information annotators need to complete the task
- **De-identification:** Remove PHI and PII before annotation where possible
- **Consent:** Where real user data is annotated, ensure appropriate consent or lawful basis
- **Access controls:** Limit which annotators can access sensitive data based on role and clearance

Synthetic Data as an Alternative

For tasks where real data carries privacy risks, synthetic data generation can create annotation-ready datasets without exposing sensitive information. For clinical NLP, for example, synthetic EHR text can provide realistic training data for de-identification models without exposing actual patient records.

Adversarial Annotation and Data Poisoning

HITL systems create an attack surface: if an adversary can influence the annotation process, they can influence the model's behavior.

Data poisoning via annotation: An attacker with access to the annotation workforce (e.g., a compromised crowdworker account) can inject systematically mislabeled examples. This is particularly effective in active learning settings, where the model preferentially queries uncertain examples — which may be the adversary's target.

Reward hacking via feedback: In RLHF, annotators (or AI-generated annotations) that consistently rate certain types of content highly can steer the model toward that content, regardless of its true quality.

Mitigation: Multiple independent annotators per item; outlier detection on annotation patterns; monitoring for anomalous agreement or disagreement; maintaining evaluation sets that cannot be influenced by the annotation workforce.

Institutional Ethics

IRB and Ethical Review

Research projects involving human subjects — including annotation workers — often require Institutional Review Board (IRB) approval. Annotation projects that collect data about workers' beliefs, responses to sensitive content, or demographic information should be reviewed under the same ethical framework as other human subjects research.

Data Governance

Organizations should have clear policies for: - What data can be sent for external annotation vs. annotated internally - How long annotation data is retained and by whom - Who has access to annotations and the models trained on them - How to handle requests to delete annotated data (GDPR right of erasure)

Transparency and Accountability

Users affected by ML systems have a legitimate interest in understanding how those systems were built. Documenting annotation methodology — who labeled the data, under what conditions, with what guidelines — is a form of accountability that benefits users, regulators, and the field as a whole.

Datasheets for Datasets {cite}`gebru2021datasheets` provides a structured template for documenting dataset provenance, annotation procedures, and known limitations.

> **See also:** Algorithmic fairness framework: {cite}`barocas2019fairness`. Ghost work and platform labor: Gray & Suri (2019). Datasheets for Datasets: {cite}`gebru2021datasheets`. Content moderation worker welfare: {cite}`newton2019trauma`. Annotator demographics and NLP datasets: {cite}`geva2019annotator`.

Future Directions

The field of human-in-the-loop machine learning is changing rapidly. Foundation models, LLMs-as-annotators, and new paradigms for human-AI collaboration are reshaping the economics and practice of HITL in ways that were not imaginable even five years ago. This final chapter maps the landscape of emerging directions and open problems.

Foundation Models and the Changing Role of Annotation

Foundation models — large models pre-trained on broad data that can be adapted to downstream tasks — are fundamentally changing the economics of HITL.

Reducing Annotation Burden

A task that previously required thousands of labeled examples for training from scratch may require only dozens when fine-tuning a foundation model. Few-shot prompting may eliminate the need for fine-tuning entirely for some tasks.

Implication: The ROI of annotation changes. Annotation effort is now better directed toward: - High-quality evaluation sets (to measure whether the adapted model actually works) - Hard cases and edge cases that the foundation model handles incorrectly - Task-specific examples that teach the model something it cannot infer from pre-training

New Forms of Specification

When the base model already understands language, code, and common sense, users can communicate with it in natural language rather than through labeled examples. A user who wants a text classifier no longer needs to label 500 examples — they can write a description of the classification task and evaluate zero-shot performance.

This shifts the HITL challenge from *example collection* to *task specification*: helping users articulate precisely what they want in a form the model can act on. This is harder than it sounds — natural language descriptions of tasks are often ambiguous in ways that annotated examples are not.

LLMs as Annotators

One of the most significant developments of 2023–2025 is the use of **LLMs as annotation oracles**. Instead of asking a human "Is this review positive or negative?", we ask GPT-4 or Claude. For well-defined classification tasks, LLM annotations often match or closely approach crowd-worker accuracy {cite}`gilardi2023chatgpt`, and the per-annotation cost of API calls is

typically orders of magnitude lower than human labor rates. Note that this applies specifically to *annotation* tasks (assigning labels), which is distinct from *evaluation* tasks (judging which of two outputs is better) — the latter is the domain of LLM-as-judge methods and has different reliability characteristics.

Where LLM Annotation Works Well

- Simple, well-defined classification tasks (sentiment, topic, intent)
- Tasks where human labels encode cultural consensus that the LLM has absorbed
- Tasks where annotation is consistent across contexts (not highly subjective)
- Generating first-pass annotations for human review

Where LLM Annotation Fails

- Highly specialized domain tasks requiring rare expertise (medical, legal, scientific)
- Tasks that require local cultural knowledge or language variation
- Safety and harm tasks, where the LLM may be miscalibrated in the same direction as the data it was trained on
- Novel task types not well-represented in pre-training

RLAIF and Constitutional AI

As discussed in Chapter 6, AI-generated feedback can be used to train reward models and guide RL fine-tuning. This creates a feedback loop: LLMs generate data, models are trained on it, and better models generate better data. The question of how to bootstrap this loop without encoding systematic errors from the initial model is a central open problem in scalable oversight research.

Weak Supervision at Scale

Programmatic labeling via labeling functions (Chapter 9) allows subject matter experts to express their knowledge as code rather than as labeled examples. Systems like Snorkel {cite}`ratner2017snorkel` and its successors have matured substantially and are now used in production at major technology companies.

Frontier directions: - **LLM-augmented labeling functions:** Use LLMs to generate labeling functions from natural language descriptions - **Slice-based learning:** Identify critical subsets of the data (slices) where labeling functions disagree and direct human annotation there - **Uncertainty-aware aggregation:** Better statistical models for combining labeling functions with different accuracies and correlations

---

Continual Learning with Human Oversight

Most ML systems are trained offline and deployed as static models. The real world changes; models that were accurate at deployment time degrade as the data distribution shifts.

Continual learning — the ability to learn from new data while retaining old knowledge — is an active research area. Human oversight is critical: automated continual learning without human review can rapidly encode distribution shifts that are errors rather than genuine changes in the world.

HITL continual learning involves: - Monitoring for distribution shift (automated) and routing shifted examples to human review - Selective retraining: human-approved examples from the new distribution are added to training data - Human review of model behavior changes after each update

---

Multi-Modal HITL

As AI systems become multi-modal — processing and generating text, images, audio, and video simultaneously — annotation becomes more complex. A single piece of content may require annotation across modalities, with dependencies between them.

Emerging task types: - Video + transcript annotation (what is happening, who is speaking, what does the text describe visually?) - Medical image + clinical report annotation - Robotic trajectory annotation (linking sensor data to action sequences)

Multi-modal foundation models (GPT-4V, Gemini, Claude) are changing the annotation landscape here too — models can now interpret images and generate candidate annotations, which humans review.

Scalable Oversight

A fundamental open problem in HITL ML is **scalable oversight** {cite}`irving2018ai,bowman2022measuring`: as AI systems become more capable than humans in specific domains, how do we maintain meaningful human oversight?

For tasks like protein structure prediction, legal analysis, or mathematical proof verification, even expert annotators may not be able to judge which of two AI outputs is correct. Current HITL methods break down when human judgment is less reliable than the model being evaluated.

Proposed approaches {cite}`bowman2022measuring`:

Debate: Two AI systems argue for different conclusions; a human judge evaluates the arguments rather than the conclusions directly. The correct conclusion should be more defensible.

Amplification: Human judges consult AI assistants (the model itself) to help evaluate complex outputs. This leverages AI capabilities to extend human oversight rather than replacing it.

Process supervision: Instead of evaluating the final output, humans evaluate the *reasoning process* — flagging flawed steps in a chain of thought, regardless of whether the final answer happens to be correct.

The Evolving Division of Labor

The long-term trajectory of HITL ML is toward a more fluid collaboration between humans and AI, where neither is simply the "labeler" and the other the "learner," but both contribute to a shared cognitive process.

Trends to watch: - **AI-assisted annotation:** AI proposes; humans decide. Quality improves as AI proposes better options. - **Human-guided exploration:** Humans set goals and constraints; AI explores the space of solutions. - **Collaborative evaluation:** Humans and AI jointly evaluate complex outputs through dialogue. - **Preference learning at scale:** Implicit signals (how users interact with AI outputs) feed continuous preference learning without requiring explicit annotation sessions.

The question of how much to trust AI judgment versus human judgment — and in what domains, at what capability levels, with what safeguards — is ultimately a societal question, not just a technical one. HITL ML provides the technical infrastructure for answering it carefully, rather than by default.

Open Research Problems

We close with a list of important open problems that the field is actively working on:

1. **Optimal stopping in active learning:** When is the marginal value of the next annotation below the cost? Principled stopping rules remain elusive.
2. **Annotation budget allocation across tasks:** In multi-task settings, how should a fixed annotation budget be split across tasks?
3. **Distribution shift in active learning:** Active learning creates biased labeled sets. How can models trained this way be properly calibrated?
4. **Reward model generalization:** RLHF reward models may fail to generalize to novel prompt-response pairs. How can we build more robust preference models?
5. **Scalable oversight:** How do we maintain human oversight as AI systems exceed human performance in specific domains?
6. **Annotator modeling:** Better statistical models for annotator behavior that capture not just competence but also systematic biases, topical expertise, and fatigue.
7. **Evaluation of alignment:** We lack ground-truth tests for whether a model is aligned with human values. Developing such tests — analogous to adversarial examples for robustness — is an open problem.
8. **Fair data labor:** Economic and institutional structures that ensure annotation workers are fairly compensated and protected, while maintaining the cost-effectiveness of large-scale annotation.

> *The goal is not to replace human judgment with machine judgment, but to build systems where the combination of both is better than either alone.*

The tools and techniques in this handbook — annotation, active learning, RLHF, preference learning, and all the rest — are means to that end. As the field evolves, the specific techniques will change. The underlying aspiration — to build systems that are both capable and genuinely aligned with human intent — will remain.

> **See also:** Scalable oversight and debate: {cite}`bowman2022measuring`. Snorkel weak supervision: {cite}`ratner2017snorkel`. For the broad future of human-AI collaboration, see {cite}`amershi2019software`.

Bibliography

Table 1: style: unsrtalpha

jupytext:
formats: md:myst
text_representation:
extension: .md
format_name: myst
kernelspec:
display_name: Python 3
language: python
name: python3

Case Study: Limen — A Human in the Loop of Everything

:::{admonition} Note on this chapter :class: important This chapter is a **speculative design essay**, not an empirical study or peer-reviewed contribution. It describes an envisioned system architecture — Limen, a voice-first AI-native desktop OS — as a worked example of how HITL principles from the preceding chapters might compose into a coherent whole. The claims here are design arguments, not experimental results. They should be read as motivated design rationale, not as validated engineering findings. :::

Every chapter in this handbook has described HITL ML as a design philosophy: a set of principles for making human involvement in learning systems deliberate, efficient, and honest. This chapter describes what it looks like to apply that philosophy as a first principle — not to a single system, but to an entire operating environment.

Limen is a voice-first, AI-native desktop operating system built on the premise that the human is always in the loop, not as a constraint to be optimized away, but as the organizing principle around which every subsystem is designed. The name is chosen deliberately: *limen* is the Latin word for threshold. In perceptual psychology, the limen is the boundary between what is perceived and what is not. For an operating system, the limen is the boundary between human intent and machine action.

The architecture described here is not proprietary. It is a set of design decisions, each of which follows naturally from the HITL principles developed in the preceding chapters. The goal is not to document a product but to show how those principles compose — how they reinforce each other when applied consistently, and what kinds of systems become possible when the human is not an afterthought.

The Problem with Conventional OS Design

A conventional operating system is not designed for humans. It is designed for programs. The human is accommodated through a layer of abstraction — a graphical interface, a file browser, a terminal — that sits on top of a substrate built for processes, memory addresses, and file descriptors.

This design choice is historically justified. When the assumptions that produced it were made, computers were expensive, humans were cheap, and the bottleneck was computation. The right optimization target was the machine.

Those assumptions no longer hold. The bottleneck, for most users in most tasks, is not compute. It is human attention: the cost of context-switching, of searching for the right file, of constructing

the right query, of remembering where something was. The machine is fast. The human is slow. The interface should optimize for the human's side of the loop.

Conventional operating systems do not do this. They are optimized for programs, and the human's job is to speak programs' language. Limen inverts this. The programs speak the human's language. The human is in the loop, and the loop is designed to fit the human.

Architecture at the Threshold

Limen's architecture is organized around a single principle: **every interaction is an event**, and every event is an opportunity for the human to teach, correct, or confirm. The system does not wait for explicit training sessions. Learning is continuous and ambient.

The Event Layer: WID

At the foundation is **WID** (the Waldiez ID system, adapted for Limen's local-first architecture) — a causality-aware event tracking system that records not just what happened, but what caused it, and what it in turn caused.

Conventional logging asks: *what happened?* WID asks: *why did it happen, and what followed?* Each event carries a causal lineage — a chain from the triggering human action through intermediate system states to the observed outcome.

This matters for HITL learning because it solves the **credit assignment problem** at the interaction level. When a user corrects the system's behavior, WID can identify not just the immediate output that was wrong, but the chain of decisions that produced it. Corrections can be applied at the right level of abstraction: the output, the decision rule, or the upstream signal.

This is the operating-system-level equivalent of what Chapter 6 describes for RLHF: the ability to trace a reward signal back through a sequence of decisions. WID provides that tracing natively, for every interaction, without requiring the user to understand it.

:::{admonition} Causal Event Tracking as HITL Infrastructure :class: note WID's design reflects a broader principle: HITL infrastructure should make it easy to ask "why did the system do that?" — not just "what did it do?" Without causal tracing, corrections fix symptoms. With it, they can fix causes. The difference between a patch and a lesson. :::

The Perception Layer: Voice-First

Limen's primary input modality is voice, processed locally using a Whisper ONNX inference pipeline. The reasons for this choice are worth stating explicitly:

Voice is the most natural human output channel for most people. It requires no training, no physical dexterity beyond ordinary speech, and no knowledge of the system's internal organization. A user who cannot locate a file in a directory hierarchy can describe what they're looking for.

Local processing preserves privacy. Voice data does not leave the device. This matters ethically — voice is biometric data, and its collection at scale by cloud providers is a documented harm — and practically: offline operation means the system remains functional without a network connection.

Voice creates a natural feedback loop. When the system responds, the user's reaction — continuing to speak, rephrasing, saying "no, that's not right" — is itself a signal. Limen treats

these reactions as HITL feedback: evidence about whether the system's interpretation of the prior utterance was correct.

The fallback chain matters as much as the primary modality. When voice fails — in a noisy environment, for a user with a speech impairment, for an input that benefits from precision — Limen degrades gracefully to a keyboard interface and then to a structured text interface. The Grandmother Test (see Chapter 5) is not an accessibility afterthought; it is a first-class architectural constraint.

The Intelligence Layer: Multi-LLM Routing

Limen does not rely on a single language model. It routes requests through a structured decision tree based on task type, required latency, privacy requirements, and confidence:

1. **Local small model** (always first): fast, private, handles routine tasks — "open the file I was editing yesterday," "set a timer," "what's the weather"
2. **Local large model** (when small model confidence is low): slower but more capable; handles structured reasoning, code generation, complex retrieval
3. **Remote model** (when local fails and user has granted permission): the exit hatch; handled transparently with explicit user notification

This structure is not novel — it is the inference-time equivalent of the active learning query strategy described in Chapter 4. At each step, the system asks: is my current model sufficient to answer this query with acceptable confidence? If not, escalate. The escalation is a cost (latency, privacy, compute); it is incurred only when necessary.

The human is in the loop at the escalation boundary. A user who has configured Limen to never escalate to remote models has made a HITL decision — one the system respects and records. A user who approves a remote query and then says "don't ask me again for this kind of request" has provided a preference that updates the routing policy.

The Interaction Design

Immediacy

Limen is designed for immediacy in the sense that Chapter 5 defines it: the user perceives the effect of their feedback. When the user corrects a system output, the correction is applied immediately and visibly. The model does not go away and train for twenty minutes. It updates in the current session.

This requires using model architectures that support efficient online updating — adapters, prefix tuning, and retrieval-augmented generation rather than full fine-tuning. The tradeoff is explicit: online updates are noisier than batch training. Limen accepts this tradeoff because immediacy is the primary requirement: the human can always confirm, reject, or refine the update.

Intelligibility

A recurring theme in IML literature is the **intelligibility requirement**: humans can only guide a model they understand, at least approximately. Limen surfaces this in the interface: when the system makes a decision, it explains, briefly, why. Not a full reasoning trace — that would overwhelm most users — but a natural-language summary of the key factor: "I'm opening the project you worked on most recently — is that right?"

This explanation is also a question. It invites correction. It makes the model's inference visible so the user can redirect it if wrong. The explanation is not generated for aesthetic reasons; it is functional HITL infrastructure.

Consistency and Drift

A problem that arises in any long-running interactive system is **behavioral drift**: the system's behavior at time $T + n$ is subtly different from its behavior at time T, in ways neither the user nor the system explicitly chose. Corrections accumulate. Edge cases compound. The model that was aligned to the user's preferences last month may no longer be aligned today.

Limen addresses this through periodic consistency checks — the operating-system equivalent of the re-presentation technique described in Chapter 13. The system surfaces historical decisions to the user: "A few weeks ago, you asked me to do X. Is that still what you'd want?" These checks serve two functions: they catch drift, and they remind the user of preferences they may have forgotten specifying.

Limen as a HITL System

Looking at Limen's architecture through the lens of this handbook, the design decisions map directly onto the concepts developed in each part.

Part I (Foundations): Limen treats every interaction as a human–machine interaction event. There is no "non-HITL mode" — the system is always learning, always attributing, always waiting for a human to participate.

Part II (Core Techniques): Active learning manifests as the confidence-based escalation chain. Interactive ML manifests in the real-time update cycle. Annotation is implicit: every correction the user makes is a label.

Part III (Learning from Human Feedback): The preference learning described in Chapter 8 appears in the routing policy updates. When a user prefers a local model's answer to a remote one, that preference is recorded and generalizes. RLHF in a personal OS means the reward model is private, individual, and continuously updated.

Part IV (Applications): Limen is a general-purpose environment, but its design is most visible in domains where the human's judgment is irreplaceable and the cost of error is high — document drafting, task prioritization, creative work.

Part V (Systems and Practice): WID is Limen's annotation platform. It is invisible to the user in normal operation, visible when needed for debugging or transparency. The quality control mechanisms (consistency checks, confidence thresholds, escalation logs) are borrowed directly from the crowdsourcing literature.

Part VI (Ethics): The local-first, privacy-preserving design is an ethical choice, not just a technical one. The human's data does not leave their device without their explicit consent. The model that learns from a user's behavior belongs to that user.

The Deeper Point

Limen is not a product pitch. It is an argument by construction.

The argument is this: if you take the principles of HITL ML seriously — if you believe that human feedback is a signal to be understood rather than a cost to be minimized, that the human is always in the loop even when designers pretend otherwise, that alignment is an ongoing process rather than a one-time event — then you end up building something that looks like Limen.

Not necessarily Limen specifically. The specific technology stack (Tauri, Rust, Babylon.js, Whisper ONNX) is one choice among many. But the architecture — causal event tracking, local-first processing, graceful degradation, continuous preference learning, transparency as a first-class feature — follows from the principles.

The field of HITL ML has spent considerable energy describing how to put humans in the loop of specific models and specific tasks. The next question is whether we can design entire *environments* around the same principles: environments where the human is always the center, the machine is always the learner, and the loop is always open.

Limen is one answer to that question.

The threshold is not something you cross and leave behind. It is where you live.

See also: The IML principles underlying Limen's interaction design are developed in Chapter 5. The preference learning approach behind the routing policy is formalized in Chapter 8. WID's causality model draws on the event attribution literature surveyed in Chapter 14. The Grandmother Test, introduced in Chapter 5, is Limen's primary interface design constraint.

On Leaving

At some point I was enrolled in a PhD program. The research was on machine learning. The institution was real. The intent was genuine.

And then I stopped.

Not because the subject stopped mattering — it matters more now than when I started. Not because the work was too hard — this handbook is harder, in its own way, than a thesis would have been. But because somewhere between the literature review and the third revision of a paper that would be read by eleven people, I lost the thread of why any of it was worth doing.

The PhD system is a loop. You enter it as a human with questions. The loop teaches you to convert those questions into papers, and papers into citations, and citations into the appearance of progress. At no point does the loop ask whether the output is useful to anyone outside the loop. That question is, in the technical sense of the term, out of scope.

I am not the first person to notice this. I will not be the last. This is not a criticism — it is a description. The loop produces real things: rigorous methods, accumulated knowledge, trained researchers. I do not regret the time I spent inside it. But I did eventually notice that I was in it.

Leaving was not a dramatic decision. It was a slow recognition that the thing I actually wanted to build did not require permission from a committee. That the people I wanted to reach — practitioners, students, engineers making real decisions about real systems — were not waiting for a formally validated thesis. That the most honest thing I could do with what I knew was to write it down in a form someone could actually use.

So I did.

This handbook is not a PhD thesis. It has no institutional affiliation, no funding acknowledgment, no ethics board approval, no examination. It will not count toward any metric that academia uses to measure the production of knowledge.

It is also — and I say this without false modesty, because false modesty would be its own kind of dishonesty — more complete, more current, more readable, and more useful than the thesis would have been.

I am a human in the loop. I was in the loop of a PhD program, and I stepped out of it. I am now in the loop of writing about loops. You are in the loop of reading about it.

None of us chose to be in these loops entirely freely. But we can choose, sometimes, to be honest about which ones we are in, and why, and whether they are producing anything worth producing.

This book is my answer to those questions.

It is not a resignation letter. Resignation implies a job, and a job implies that someone else decided the terms. This is something else: a record of what I chose to do with the time, and a

quiet argument that the choice was the right one.

You can be the judge of that. You are, after all, a human in the loop.

— *The Author*